

UNIFIED
MODELING
LANGUAGE



Diagramme de séquence

The background is a light gray color. It is decorated with stylized, light gray leaf motifs. On the left and right sides, there are vertical stems with several leaves. Above the text, there are larger, more complex leaf shapes with internal geometric lines (diagonals and horizontals) and small circles. The overall aesthetic is clean and modern.

Rôle du diagramme de séquences

Diagramme de séquence (DS)

Les diagrammes de cas d'utilisation modélisent à **QUOI** sert le système, en organisant les interactions possibles avec les acteurs.

Les diagrammes de classes permettent de spécifier la structure et les liens entre les objets dont le système est composé : ils spécifient **QUI** sera à l'œuvre dans le système pour réaliser les fonctionnalités décrites par les diagrammes de cas d'utilisation.

Diagramme de séquence (DS)

Les diagrammes de séquences permettent de décrire **COMMENT** les éléments du système interagissent entre eux et avec les acteurs :

Les objets au cœur d'un système interagissent en s'échangeant des messages.

Les acteurs interagissent avec le système au moyen d'IHM (Interfaces Homme-Machine).

Point de vue temporel sur les interactions

Diagramme de Séquence Boîte Noire (BN)

- Appelé aussi Diagramme Séquence Système (DSS)
- **Modélise les interactions entre acteurs et Système**
- Fait pendant la capture des besoins
- Permet de décrire les scénarios des cas d'utilisation

Niveau Analyse

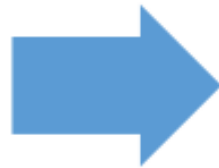


Diagramme de séquence Boîte Blanche (BB)

- **Modélise les interactions entre objets** à l'intérieur du système
- Permet de réfléchir à l'affectation de responsabilités aux objets :

- ✓ Qui crée les objets?
- ✓ Qui permet d'accéder à un objet?
- ✓ Quel objet reçoit un message provenant de l'interface?
- ✓ ...

Niveau Conception



boîte noire

vs

boîte blanche

DS Boîte Noire

Appelé aussi Diagramme de Séquence « Système » (DSS) ;
Le système informatique à développer est considéré comme une boîte noire ;
Le comportement du système est décrit vu de l'extérieur, sans modéliser le comment de sa réalisation en interne ;
On repart de la description textuelle d'un cas d'utilisation et transformer chaque étape en une flèche représentant un message.

DS Boîte Noire

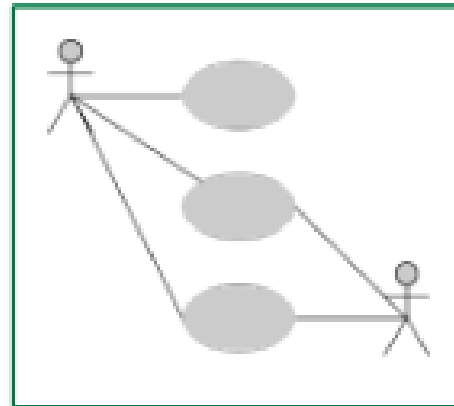
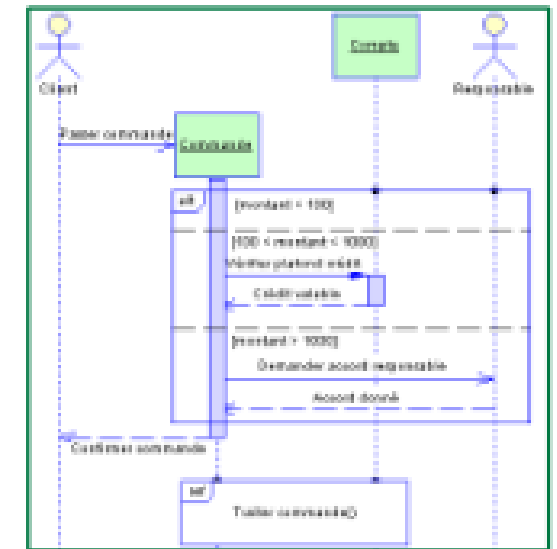


Diagramme de cas d'utilisation



Diagrammes de séquence système

DS Boîte Noire

Le système est considéré comme un tout et il est représenté par une ligne de vie ;

Chaque acteur est également associé à une ligne de vie ;

L'ordre chronologique se déroule vers le bas ;

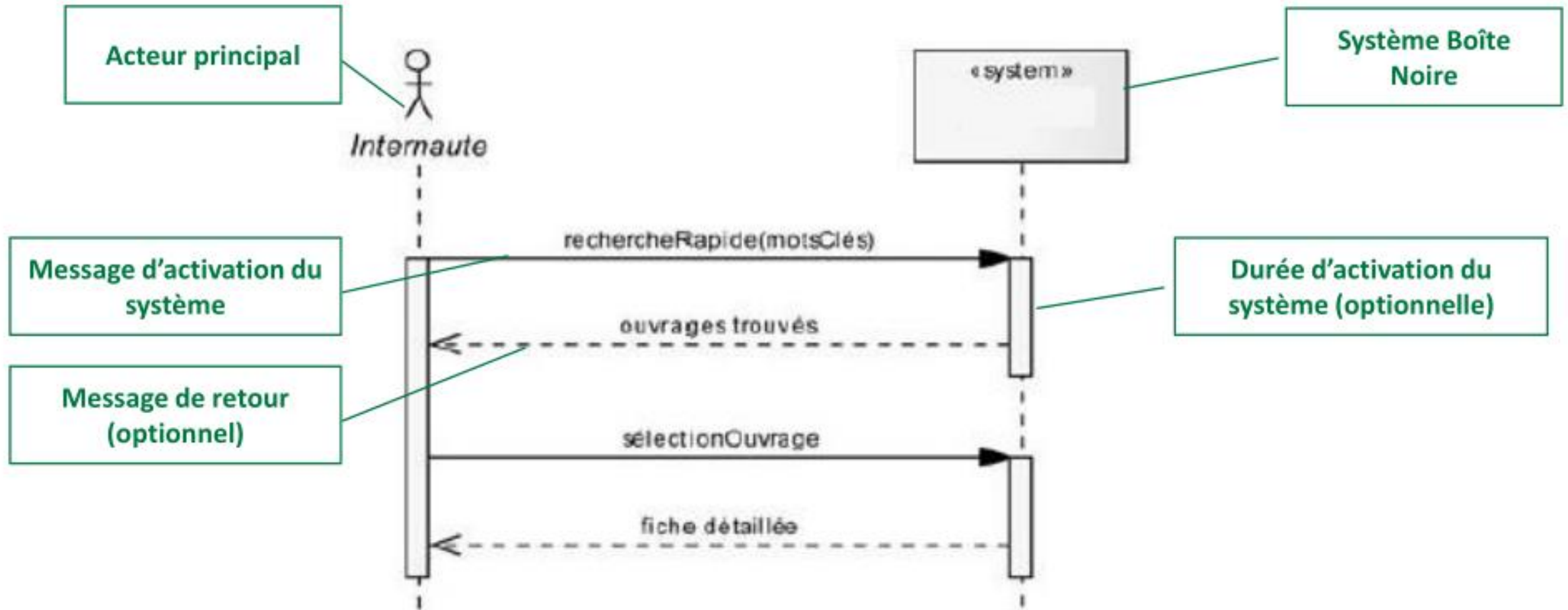
L'ordre des messages doit suivre la séquence décrite dans le cas d'utilisation.

Exemple

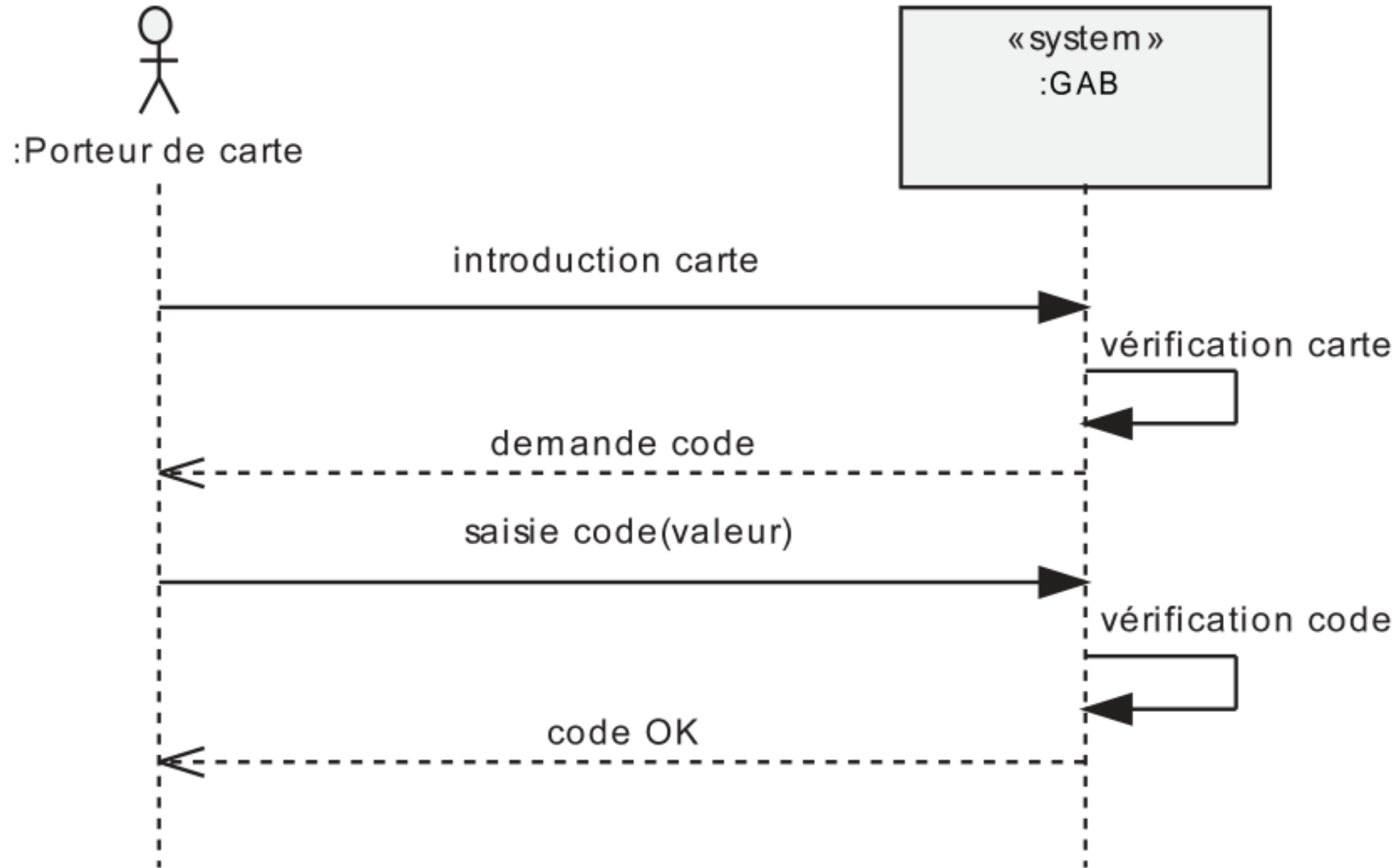
On considère un système de Librairie en ligne.

On veut modéliser le DSS du cas d'utilisation « Chercher des ouvrages » déclenché par l'acteur principal : l'Internaute

Exemple



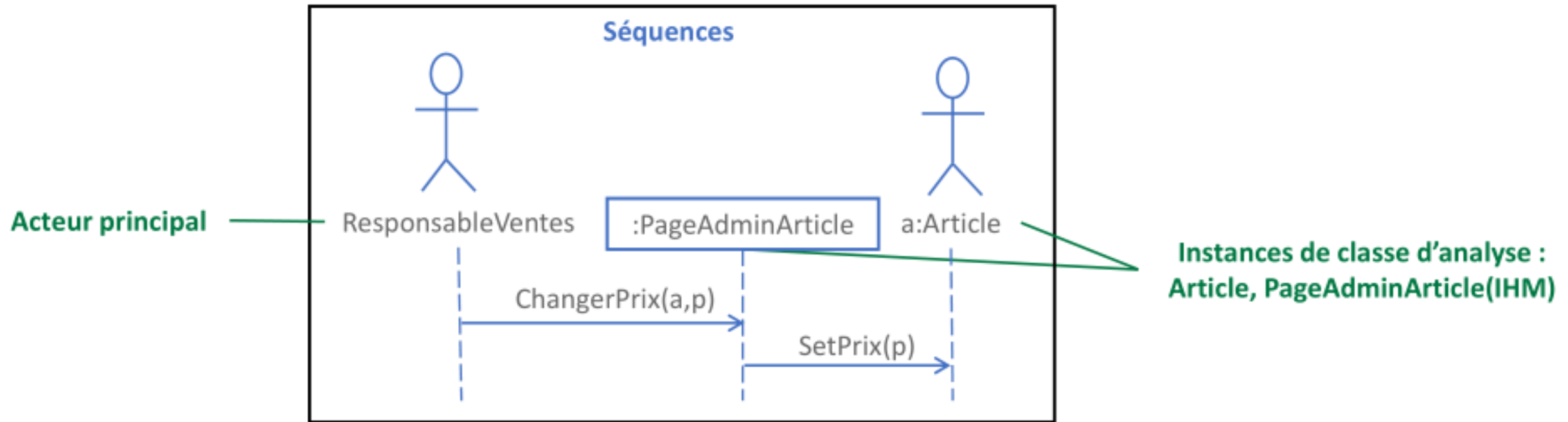
sd S'authentifier (nominal)



DS Boîte Blanche

Appelé aussi Diagramme d'interaction ;
Permet d'attribuer précisément les responsabilités de comportement aux classes d'analyse du diagramme de classes
Permet de corriger le diagramme de classes d'analyse ;
Permet de construire une première version de diagramme de classe de conception ;
Il est centré sur une collection d'objets (instances) qui collaborent (échanges des messages).

Exemple



The background features a symmetrical arrangement of stylized, light gray leaf and geometric shapes on a light gray background. On the left and right sides, there are clusters of leaves of various shapes, some with internal white lines indicating veins. Interspersed among the leaves are several small circles and larger, more complex geometric shapes, including one with a heart-like outline and internal white lines. The overall design is minimalist and modern.

boîte blanche

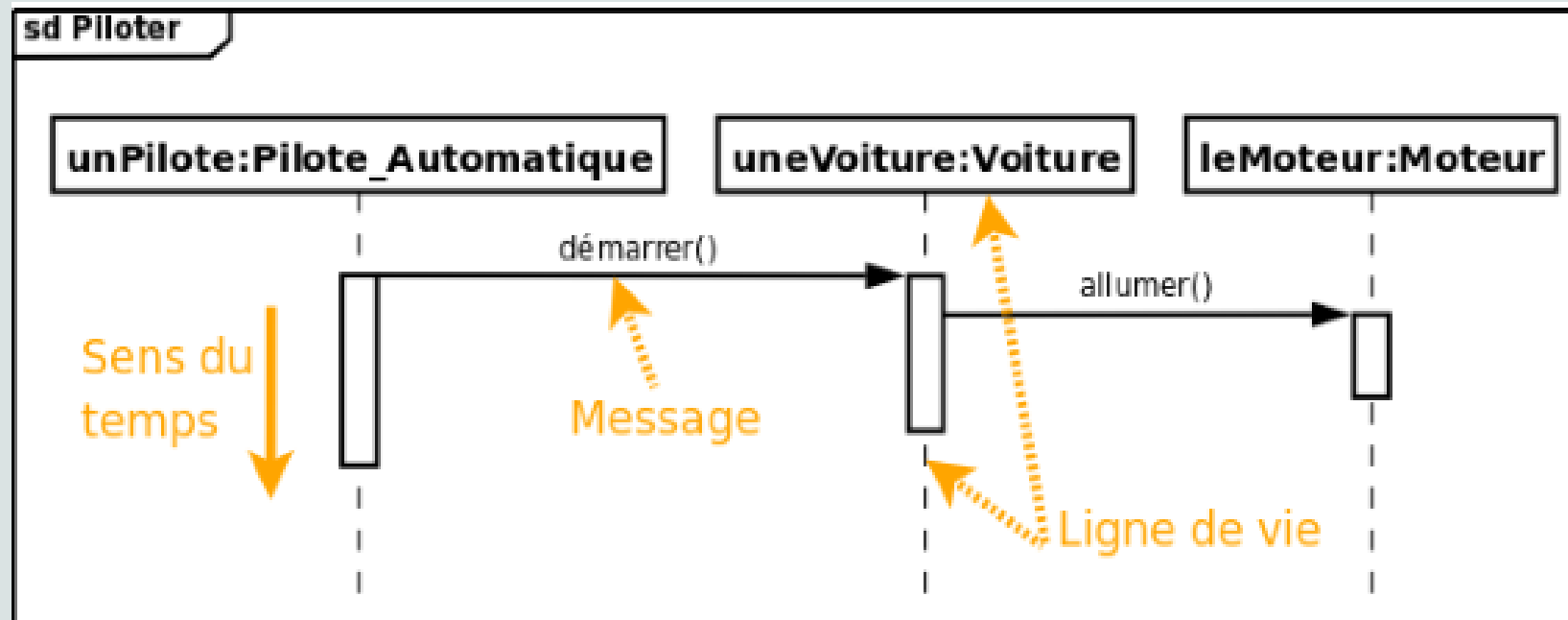
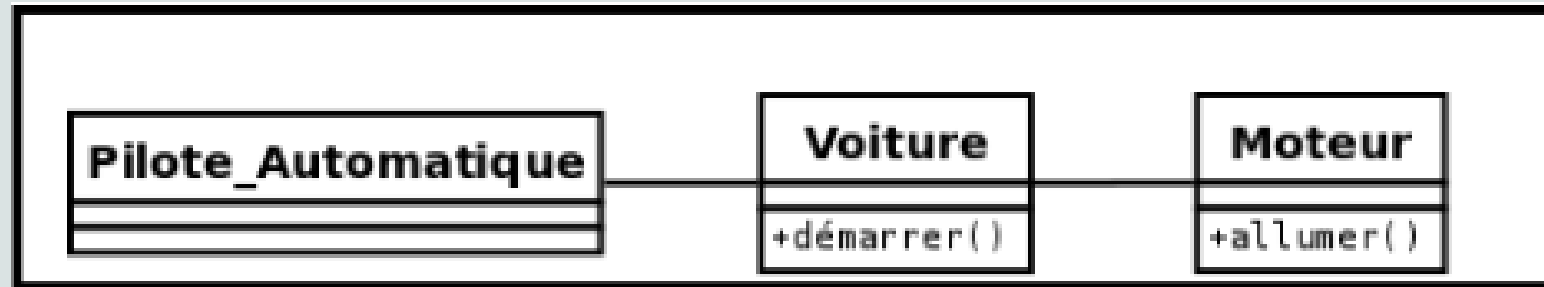
DS Boîte Blanche (Diagramme d'interactions)

Le diagramme de séquence Boîte Blanche est un diagramme d'interaction mettant l'accent sur la chronologie de l'envoi des messages.

Il permet d'établir un lien entre les diagrammes de cas d'utilisation et les diagrammes de classes : il montre comment des objets (i.e. des instances de classes) communiquent pour réaliser une certaine fonctionnalité.

Il apporte ainsi un aspect dynamique à la modélisation du système.

DS Boîte Blanche (Diagramme d'interactions)

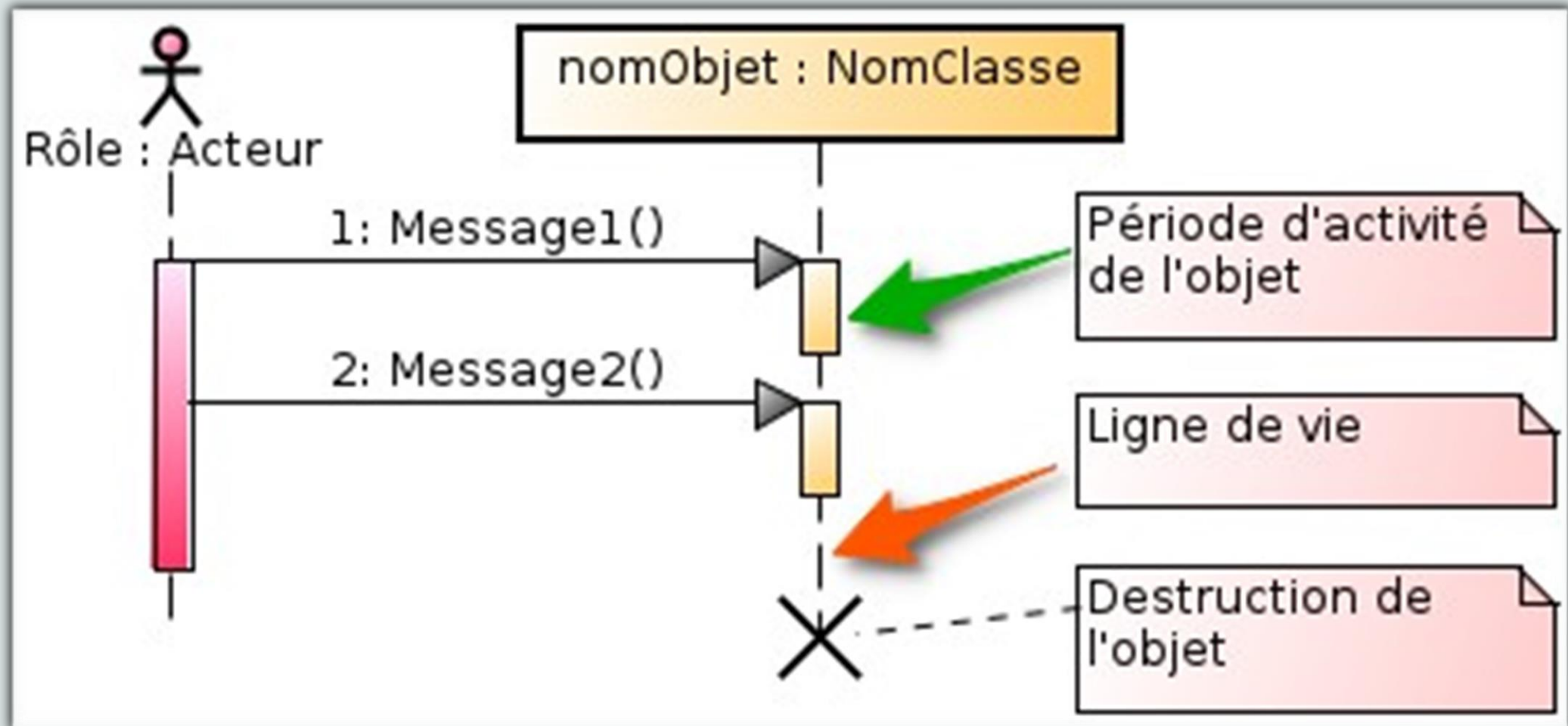


DS Boîte Blanche (Diagramme d'interactions)

Pour produire un DS Boîte Blanche, il faut se focaliser sur un sous-ensemble d'éléments du système (objets, classes) et étudier leur façon d'interagir (collaborer) pour décrire un comportement particulier.

L'interaction décrit l'activité interne des objets d'une classe (appelés lignes de vie), par les messages qu'ils échangent.

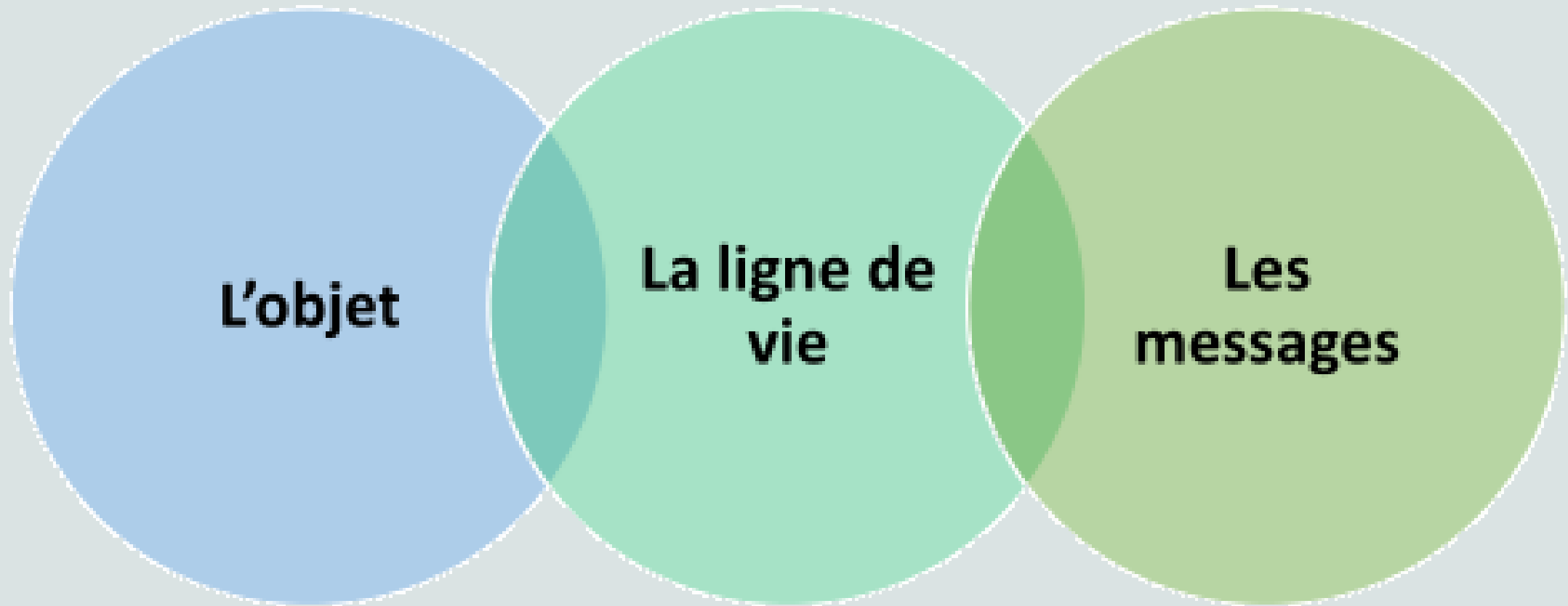
DS Boîte Blanche (Diagramme d'interactions)



The background is a light gray color. It is decorated with various stylized, light gray elements. On the left and right sides, there are vertical stems with several oval-shaped leaves. Some of these leaves have internal white lines representing veins. Above and below these stems are smaller, individual leaves and small solid circles. In the upper corners, there are larger, more complex shapes that resemble stylized flowers or leaves with internal white lines forming a grid or star pattern.

Délimitation du diagramme de séquence

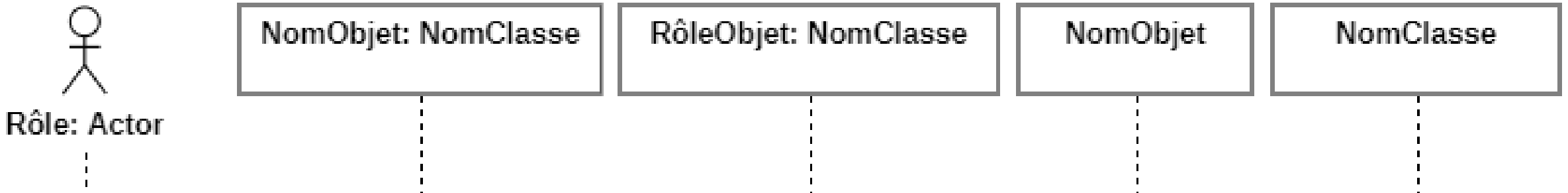
Délimitation du DS



L'objet

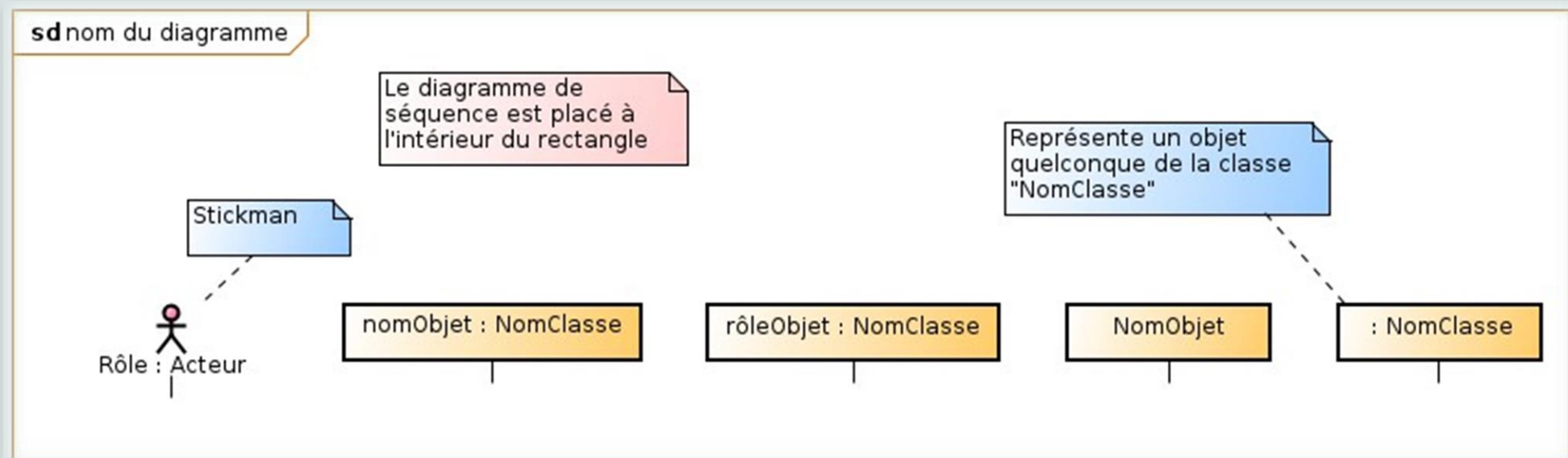
Dans un diagramme de séquence, l'objet est représenté par un rectangle dans lequel figure le nom de l'objet.

Le nom de l'objet peut prendre l'une des quatre formes suivantes :



L'objet

Les diagrammes de séquences représentant les échanges entre les objets mais aussi les échanges avec les acteurs, on trouve aussi la représentation du stickman (qui peut être considéré comme un objet).

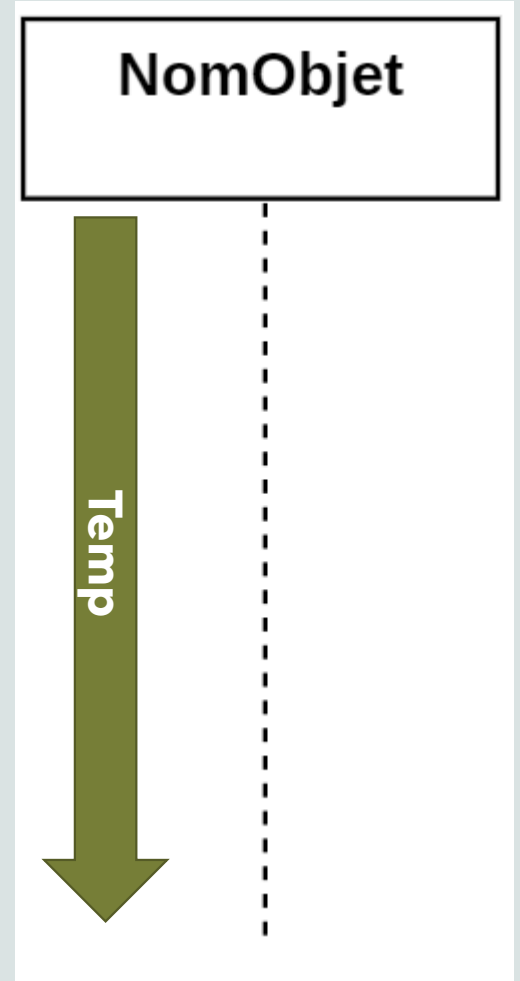


La ligne de vie

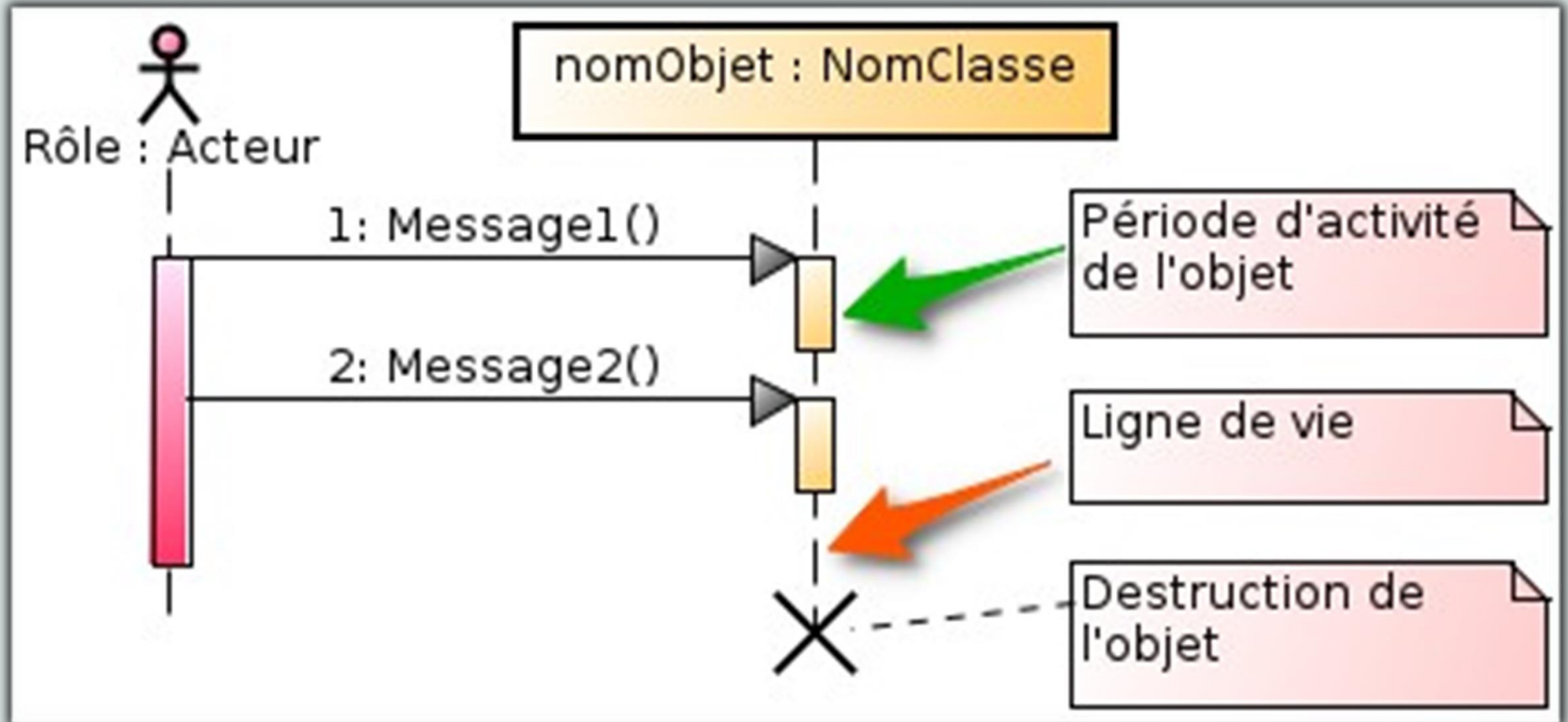
Le diagramme de séquence fait entrer en action les instances de classes intervenant dans la réalisation d'un cas d'utilisation particulier.

La ligne de vie indique les périodes d'activité de l'objet (généralement, les moments où l'objet exécute une de ces méthodes).

Lorsque l'objet est détruit, la ligne de vie s'achève par une croix.



La ligne de vie

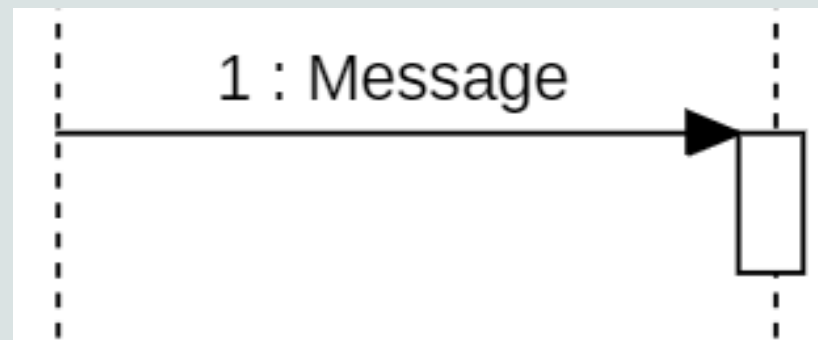


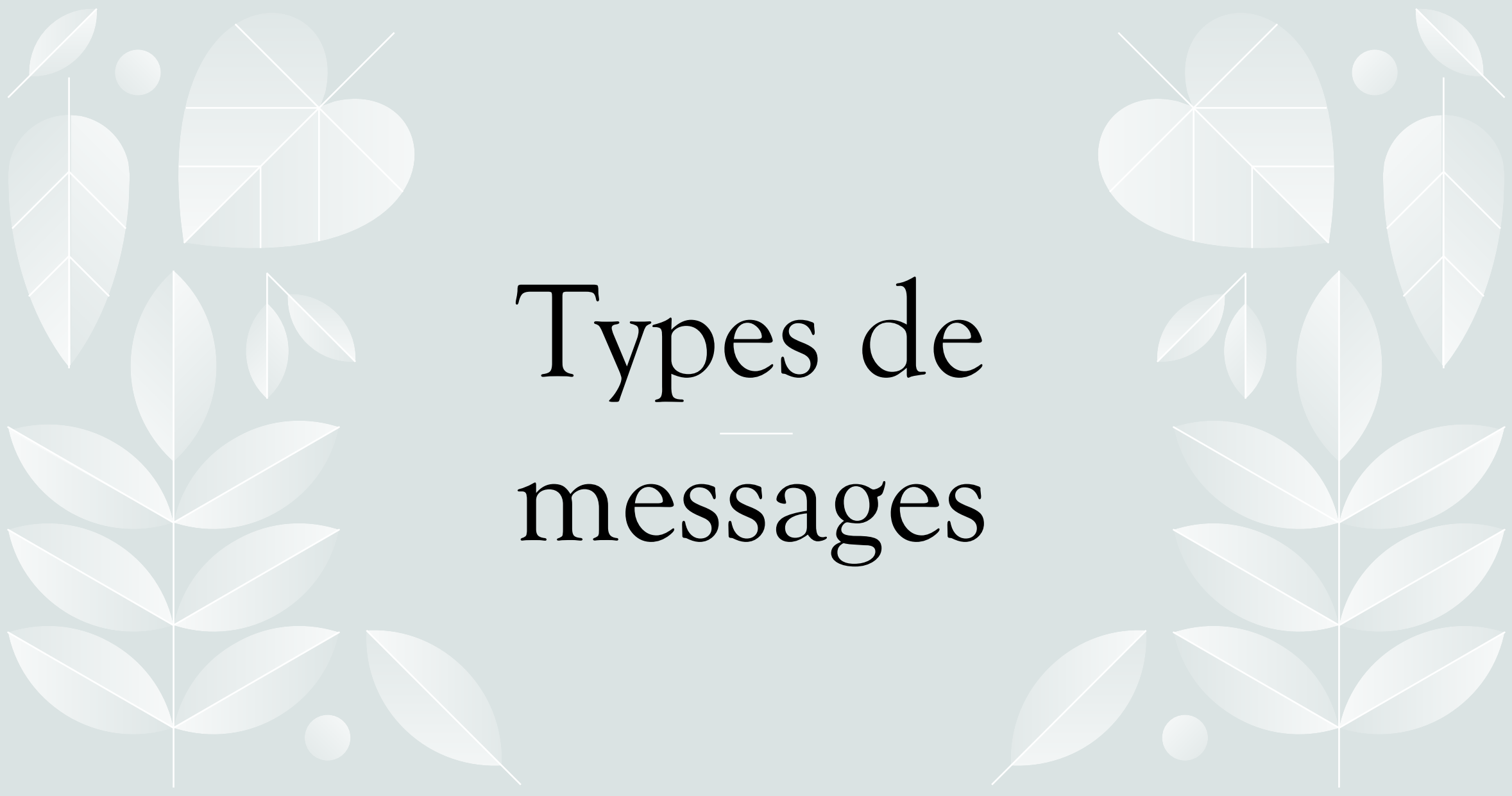
Les messages

Un message définit une communication particulière entre des lignes de vie.

Un message est une communication d'un objet vers un autre objet.

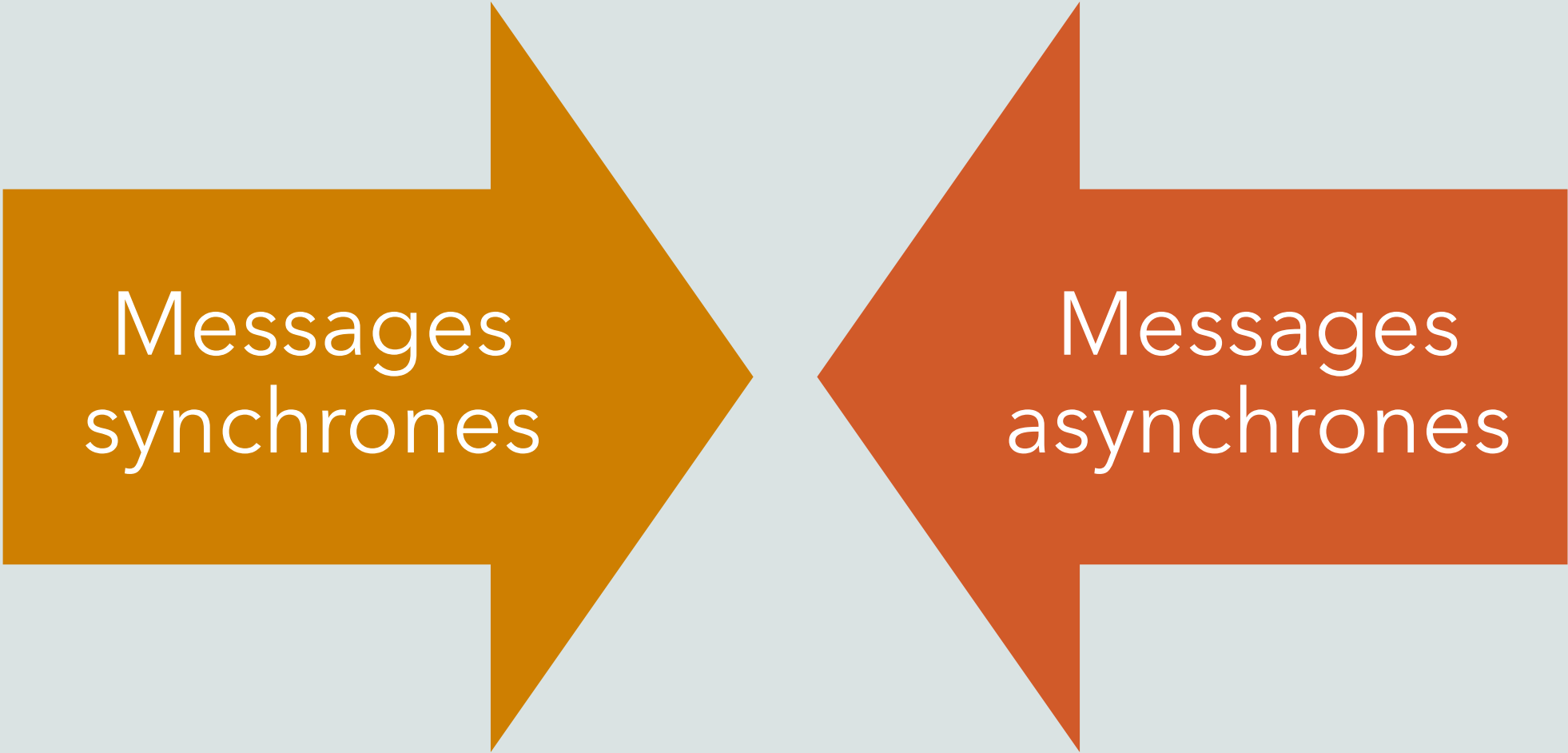
La réception d'un message est considérée par l'objet récepteur comme un événement qu'il faut traiter (ou pas).





Types de — messages

Types de messages

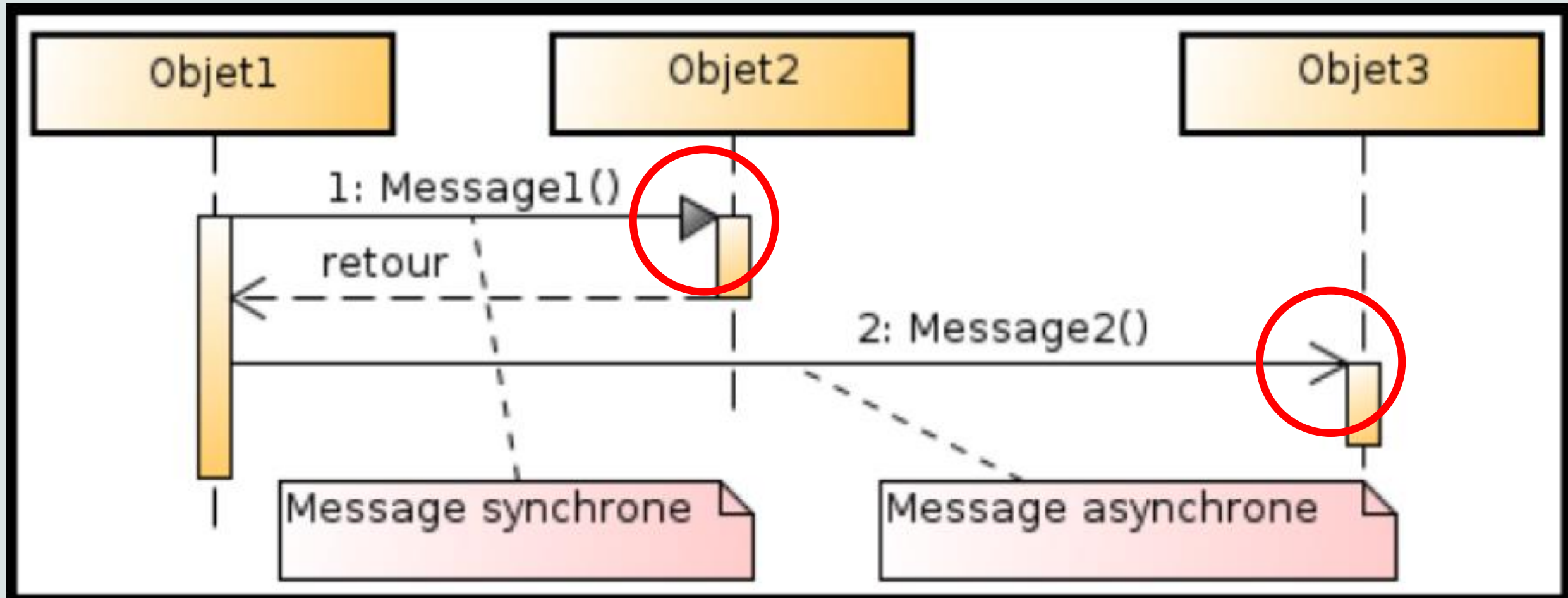


The diagram consists of two large, stylized arrows pointing towards each other. The left arrow is orange and contains the text 'Messages synchrones'. The right arrow is a darker orange and contains the text 'Messages asynchrones'. The arrows are positioned such that their tips meet in the center, creating a symmetrical composition.

Messages
synchrones

Messages
asynchrones

Types de messages



Messages asynchrones

Dans le cas d'un message asynchrone, l'expéditeur n'attend pas la fin de l'activation de la méthode invoquée chez le destinataire.

Graphiquement, un message asynchrone se représente par une simple flèche en traits pleins partant de la ligne de vie d'un objet expéditeur et allant vers celle de l'objet cible.



Un message asynchrone peut être :

- o Un appel de méthode :

Fréquent dans un système multi-threads (multi-tâche). Ainsi, l'objet expéditeur n'étant pas bloqué pendant l'exécution de la méthode, il peut continuer ainsi à envoyer d'autres messages.

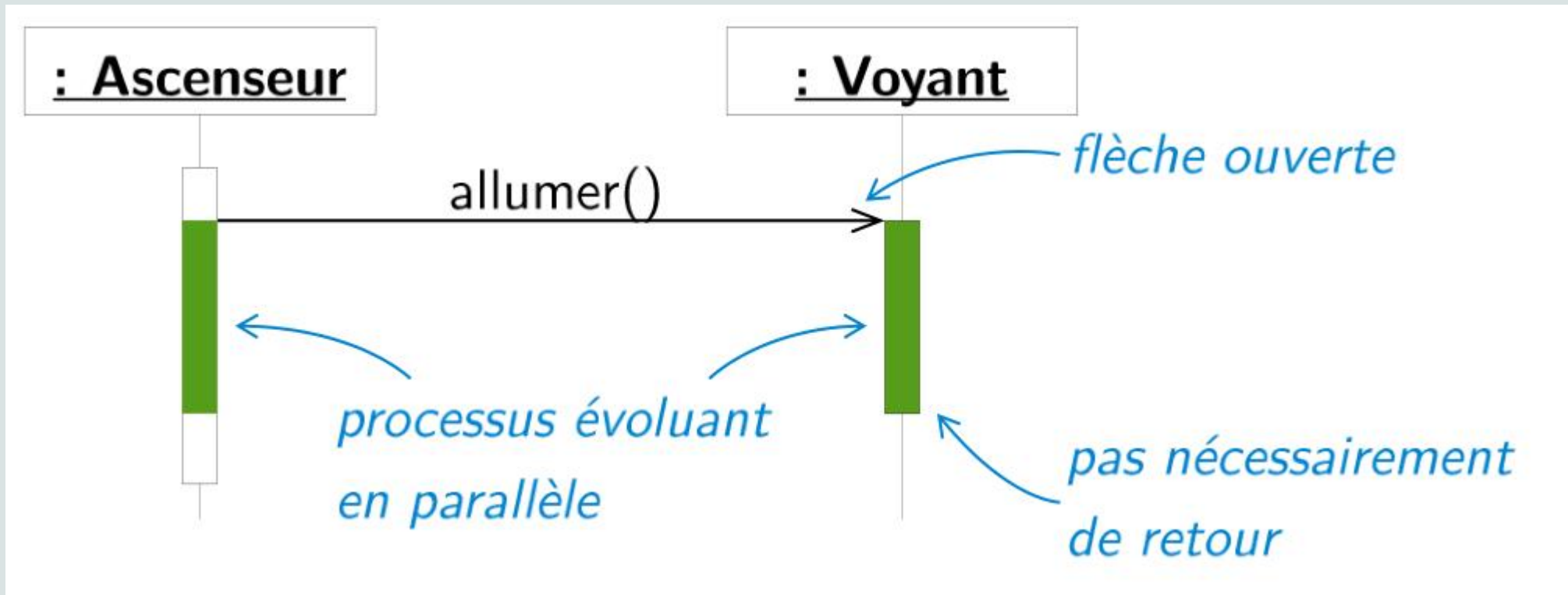
Un message asynchrone peut être :

- o Un signal (cas le plus fréquent) :

L'objet expéditeur transmet juste une information à l'objet destinataire. Souvent, ce sont les acteurs ou les périphériques qui envoient des signaux, typiquement utilisé dans la gestion événementielle d'une IHM graphique.

Exemple:

Message asynchrone Emetteur **non bloqué**, continue son exécution



Messages synchrones

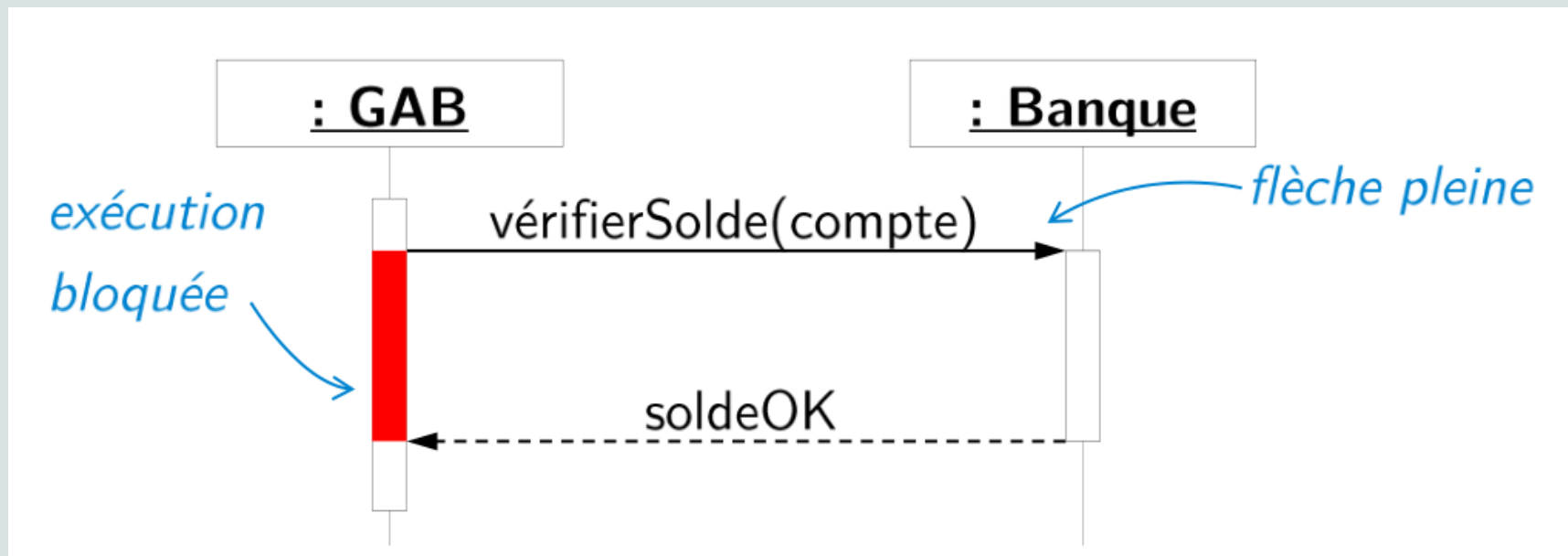
La réception d'un message synchrone doit provoquer chez le destinataire le lancement d'une de ses méthodes (qui souvent porte le même nom que le message).

L'expéditeur du message reste **bloqué** pendant toute l'exécution de la méthode et attend donc la fin de celle-ci avant de pouvoir lancer un nouveau message.

C'est le message le plus fréquemment utilisé.

Messages synchrones

Graphiquement, un message synchrone se représente par une flèche en traits pleins et à l'extrémité pleine. Ce message peut être suivi d'une réponse (message de retour) qui se représente par une flèche en pointillé.



Messages synchrones – Syntaxe

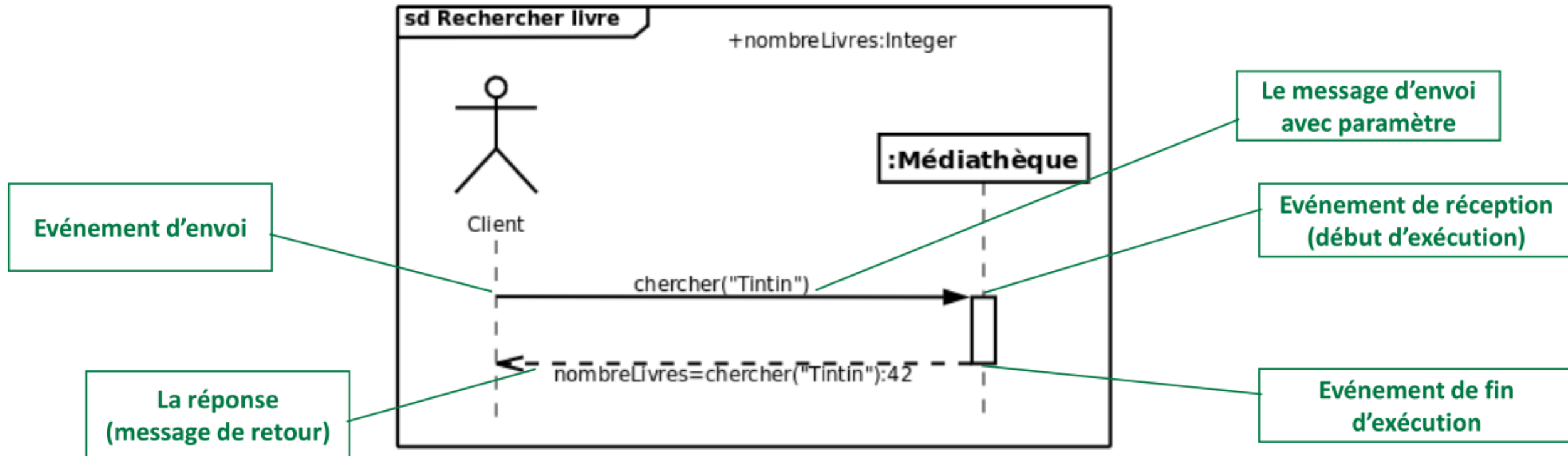
Dans la plupart des cas, la réception d'un message synchrone est suivie de l'exécution d'une méthode d'une classe. Cette méthode peut recevoir des arguments.

La syntaxe des messages permet de transmettre ces arguments : **message (p1,p2,...)**

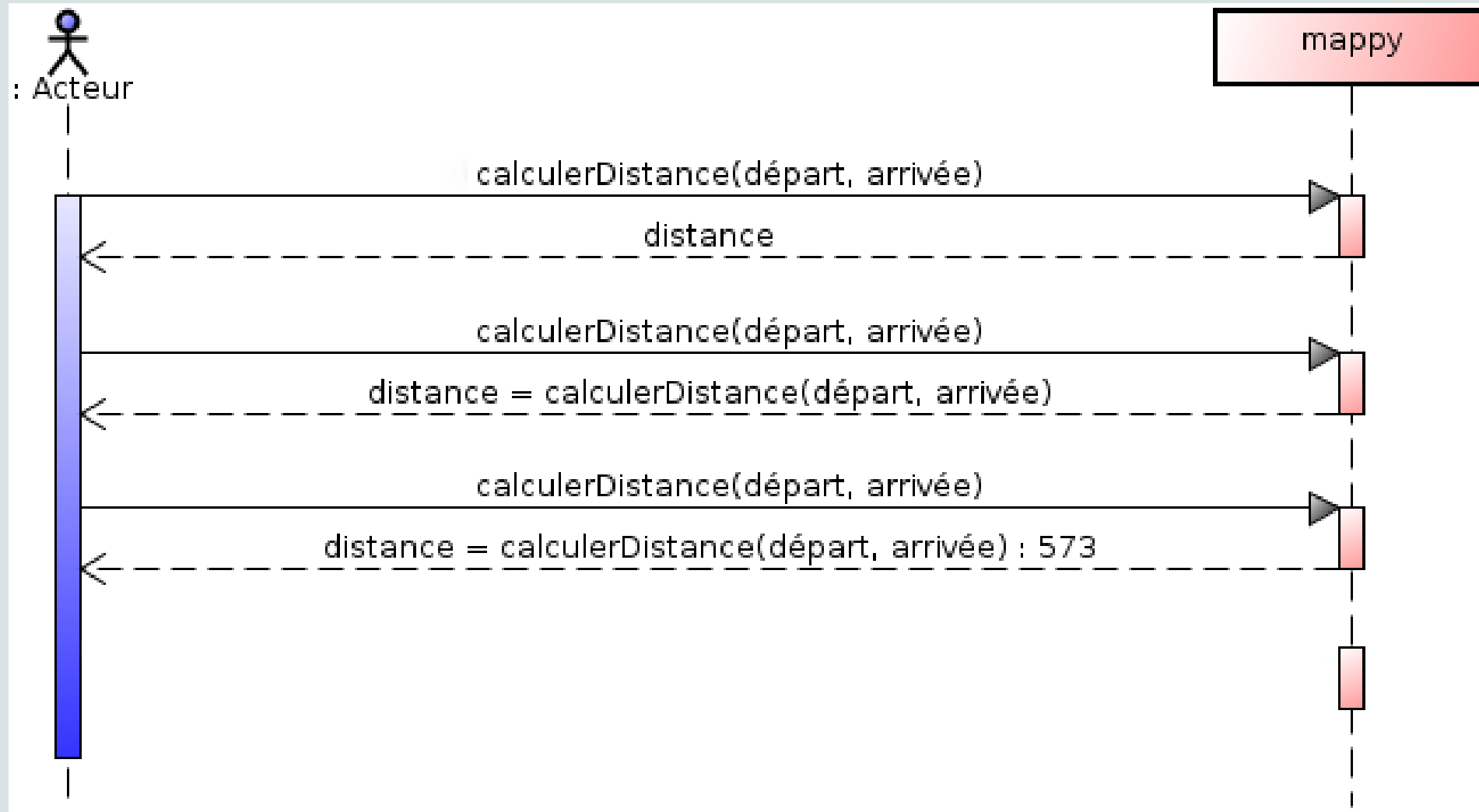
La syntaxe de réponse à un message est la suivante :

[<attribut> =] message [: <valeur_de_retour>]

Exemple



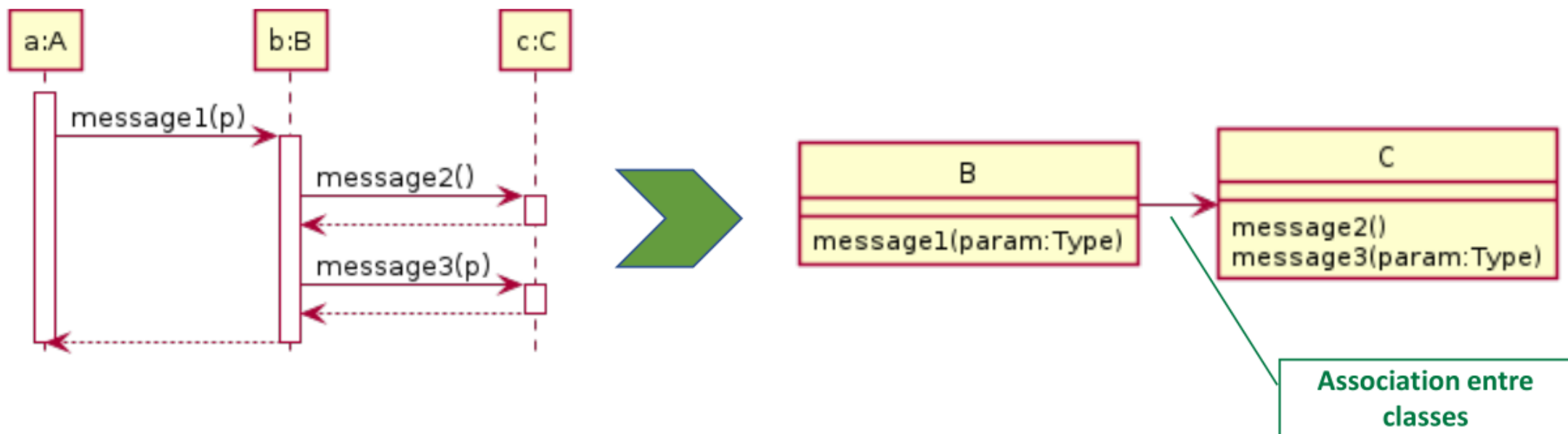
Exemple



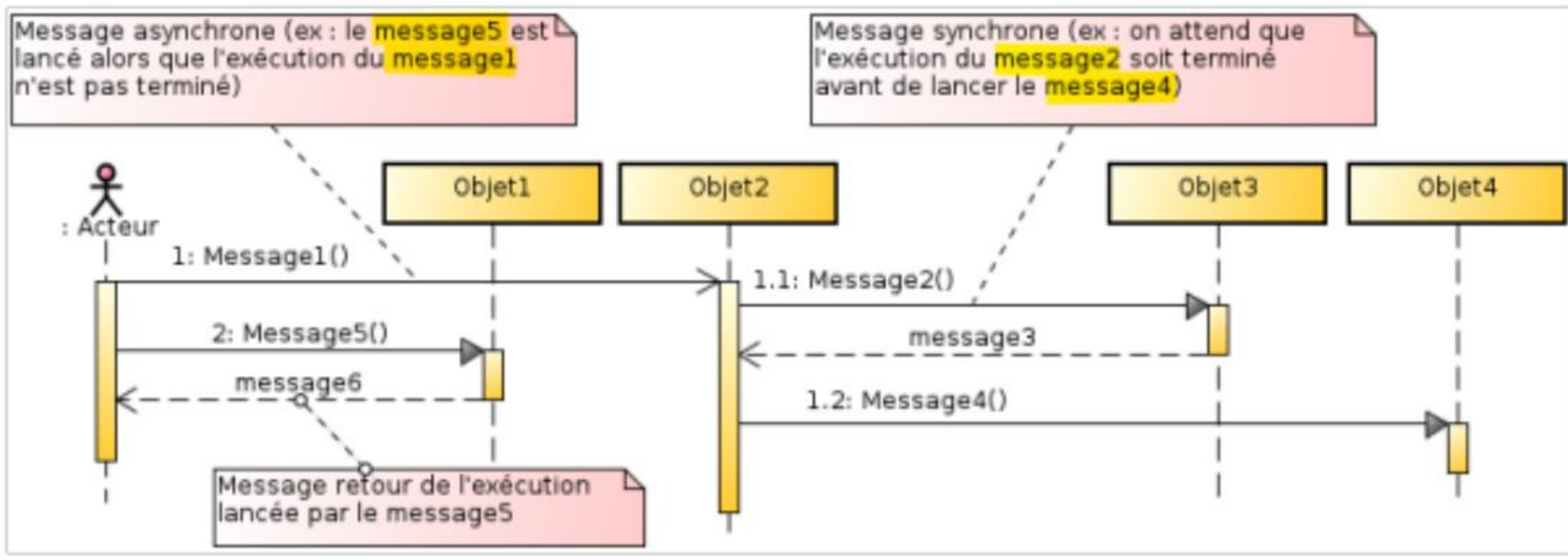
Messages synchrones et diagramme de classe

Les méthodes correspondant aux messages synchrones doivent être définies dans un diagramme de classes.

Les méthodes sont définies dans la classe du récepteur, et pas de l'émetteur du message.



Exemple



Autres types – Message perdu et trouvé

Le message perdu est un message dont on connaît l'émetteur mais pas le récepteur. Il est représenté par une flèche partante de la ligne de vie d'un élément vers un disque noir.

Le message perdu pouvant être, à l'origine, synchrone ou asynchrone,

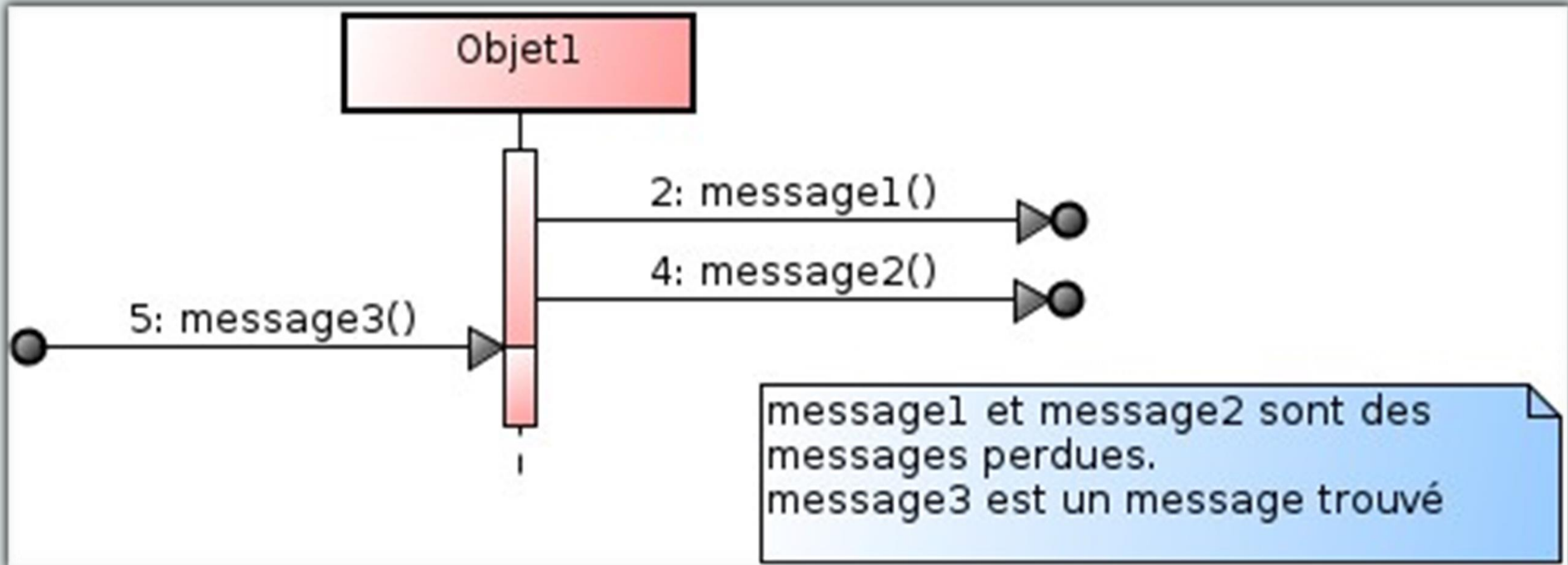
Cette sorte de message permet de modéliser, par exemple, les scénarii de pertes de message sur un réseau.

Autres types – Message perdu et trouvé

Le message trouvé est un message dont on connaît le destinataire mais pas l'émetteur. Il est représenté par une flèche partant d'un disque noir vers la ligne de vie d'un élément.

Ce message peut être utilisé pour modéliser le comportement d'un élément suite à la réception d'un message d'exception.

Autres types – Message perdu et trouvé



Autres types – Messages de création d'instance

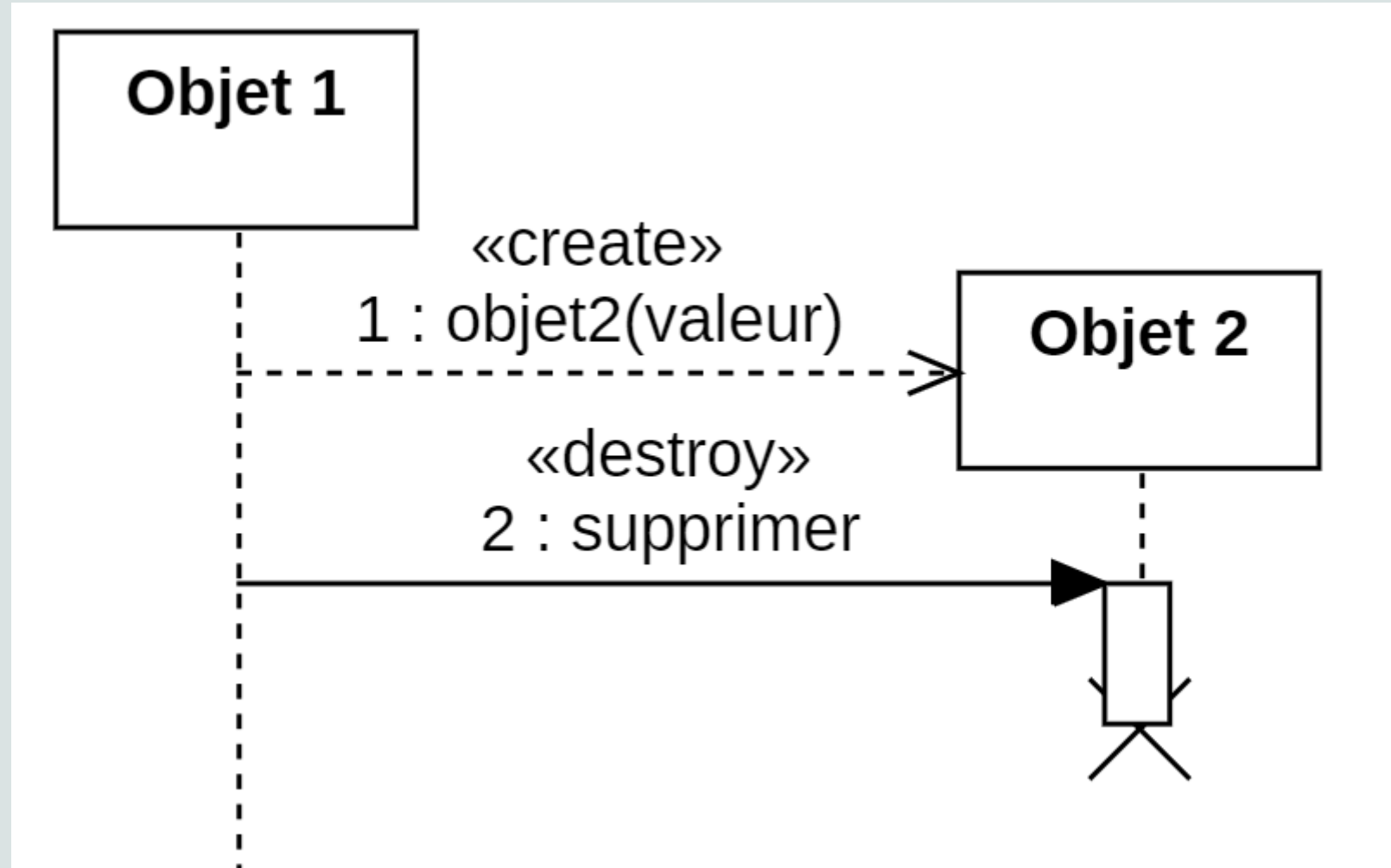
Une séquence peut aussi contenir la création ou la destruction d'un objet :

La création d'un objet est matérialisée par un message spécifique, appel d'un constructeur, généralement accompagné du stéréotype « **create** » qui pointe sur le début (le sommet) de la ligne de vie de l'objet créé (Le rectangle de l'instance de la classe est alors surbaissée).

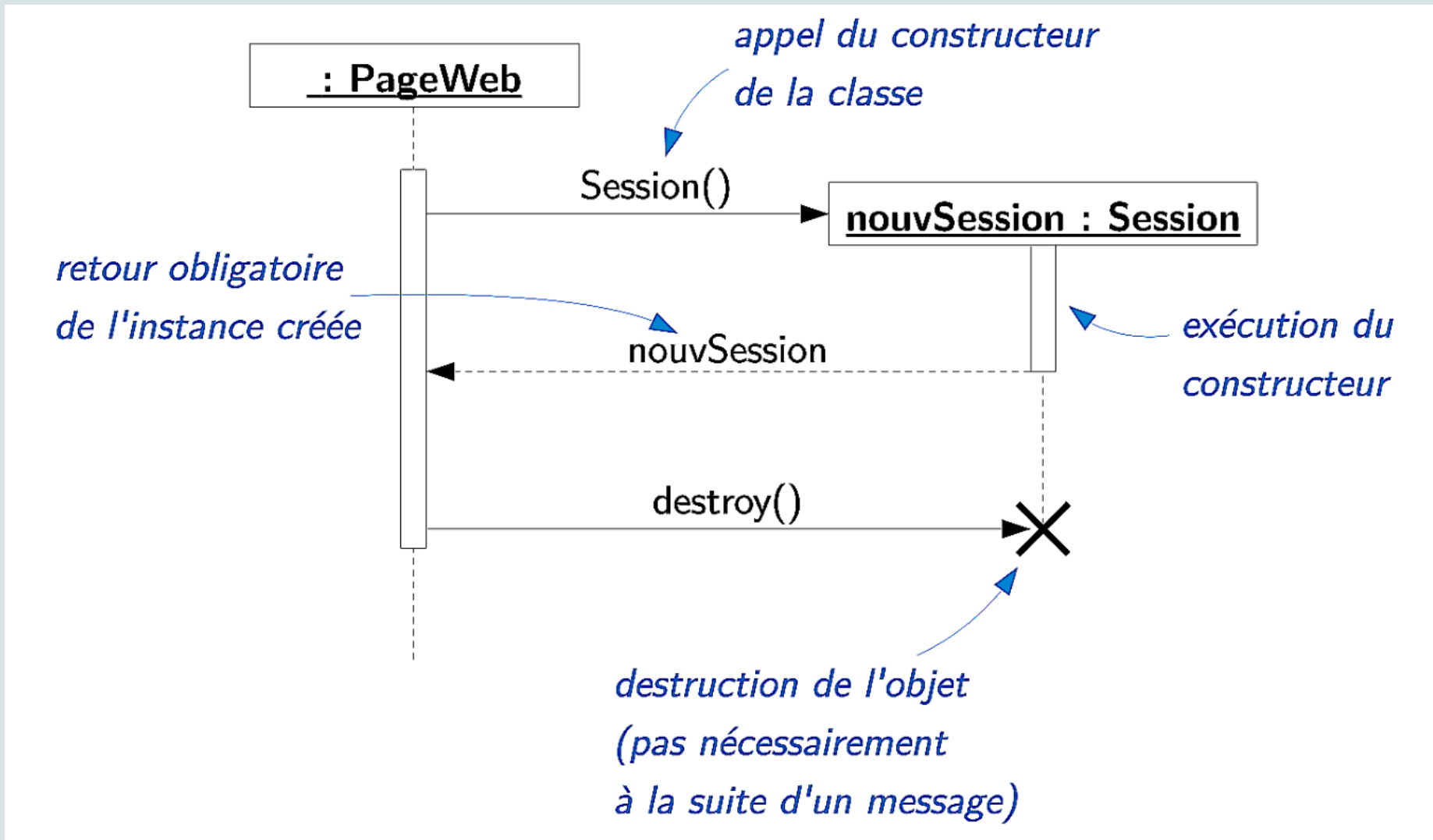
Autres types – Messages de destruction

La destruction d'un objet est représentée par une croix à la fin de sa ligne de vie. Souvent l'objet est détruit suite à la réception d'un message mais ce n'est pas obligatoire. Dans ce cas là, il porte le stéréotype « **destroy** ».

Messages de création et de destruction



Messages de création et de destruction



The background features symmetrical, stylized leaf patterns in a light gray color. On the left and right sides, there are clusters of leaves of various shapes, some with internal vein details. Small circles are interspersed among the foliage. The central text is positioned between these decorative elements.

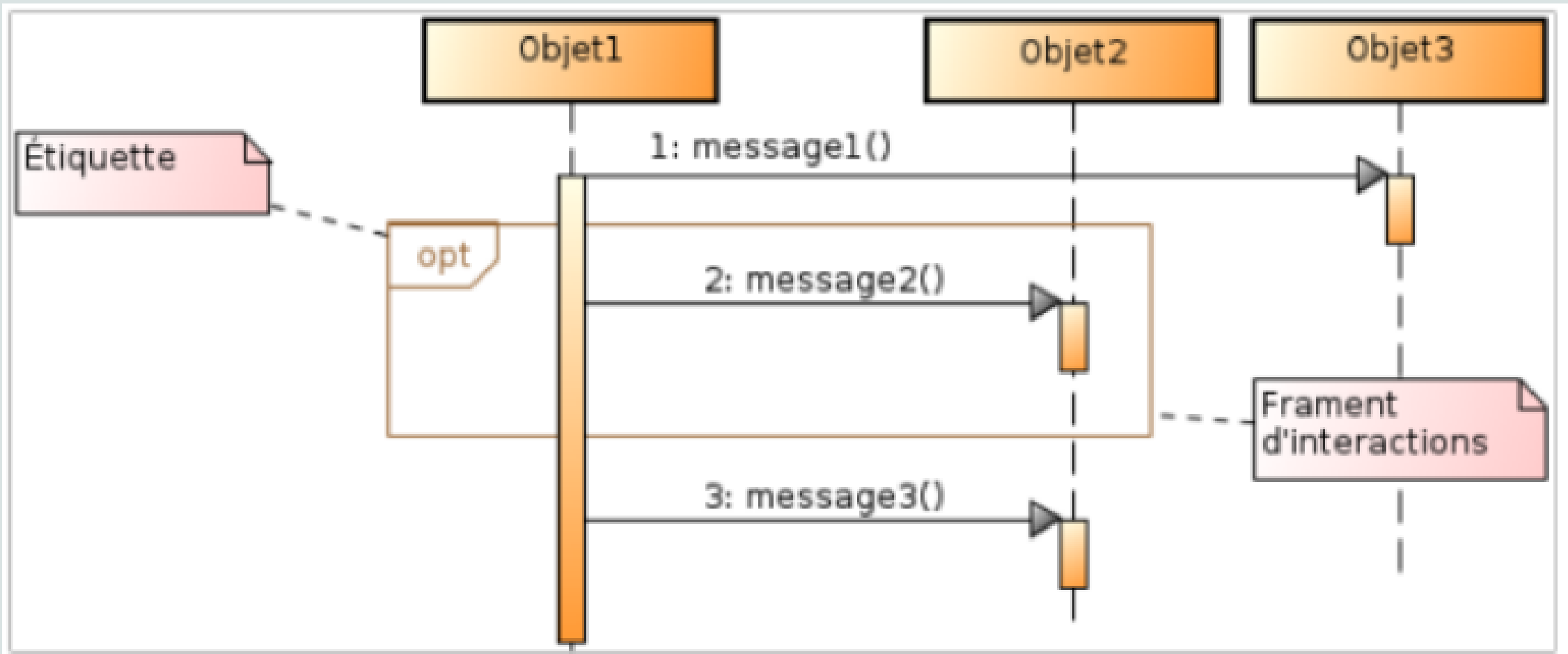
Fragments d'interactions combinés

Fragment combiné

Un fragment d'interactions est une partie du diagramme de séquence (délimitée par un rectangle) associée à une étiquette (dans le coin supérieur gauche).

L'étiquette contient un **opérateur d'interaction** qui permet de décrire des modalités d'exécution des messages à l'intérieur du cadre.

Exemple



Opérateur d'interaction

Dans l'étiquette du fragment, figure le type de la combinaison (appelé opérateur d'interaction).

Opérateurs de choix et de boucle

- Opérateur d'option « opt »
- Opérateur d'alternative « alt »
- Opérateur de boucle « loop »
- Opérateur d'interruption « break »

Opérateurs contrôlant l'envoi en parallèle de messages

- Opérateur Parallèle « par »

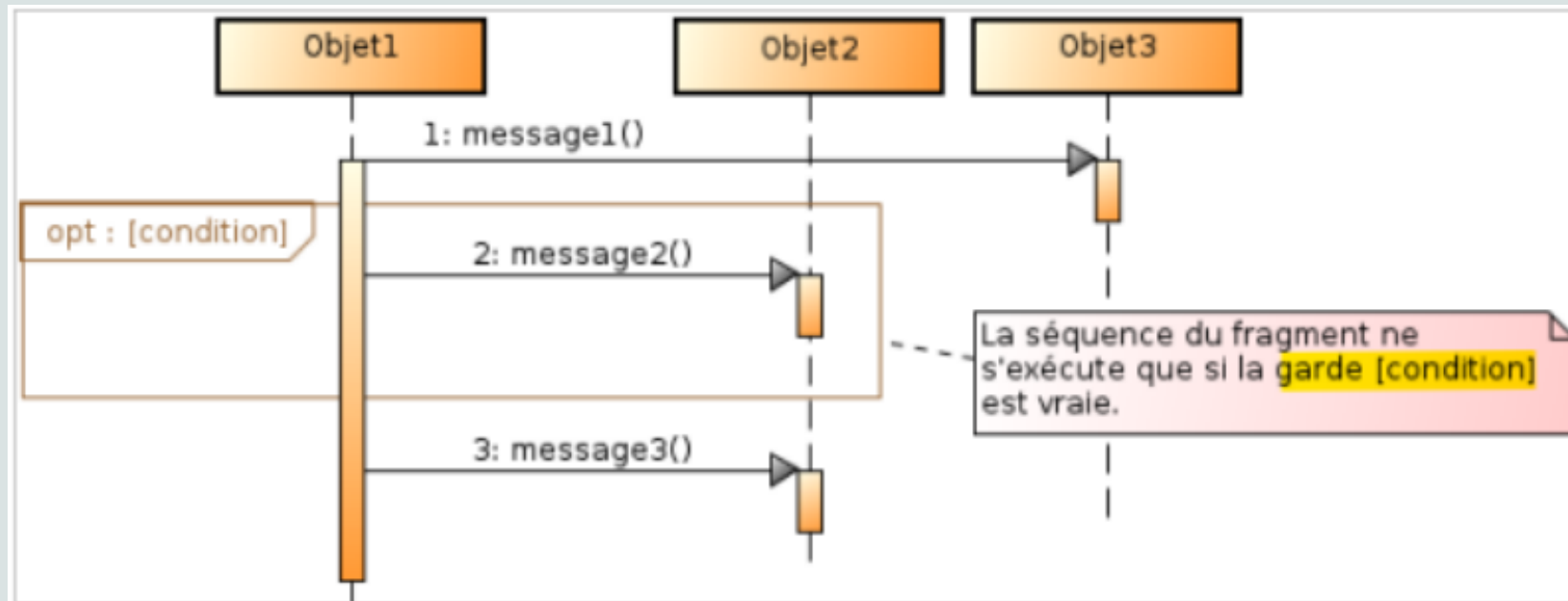
Opérateurs contrôlant l'envoi de messages

- Opérateur d'omission « ignore »
- Opérateur d'affirmation « assert »

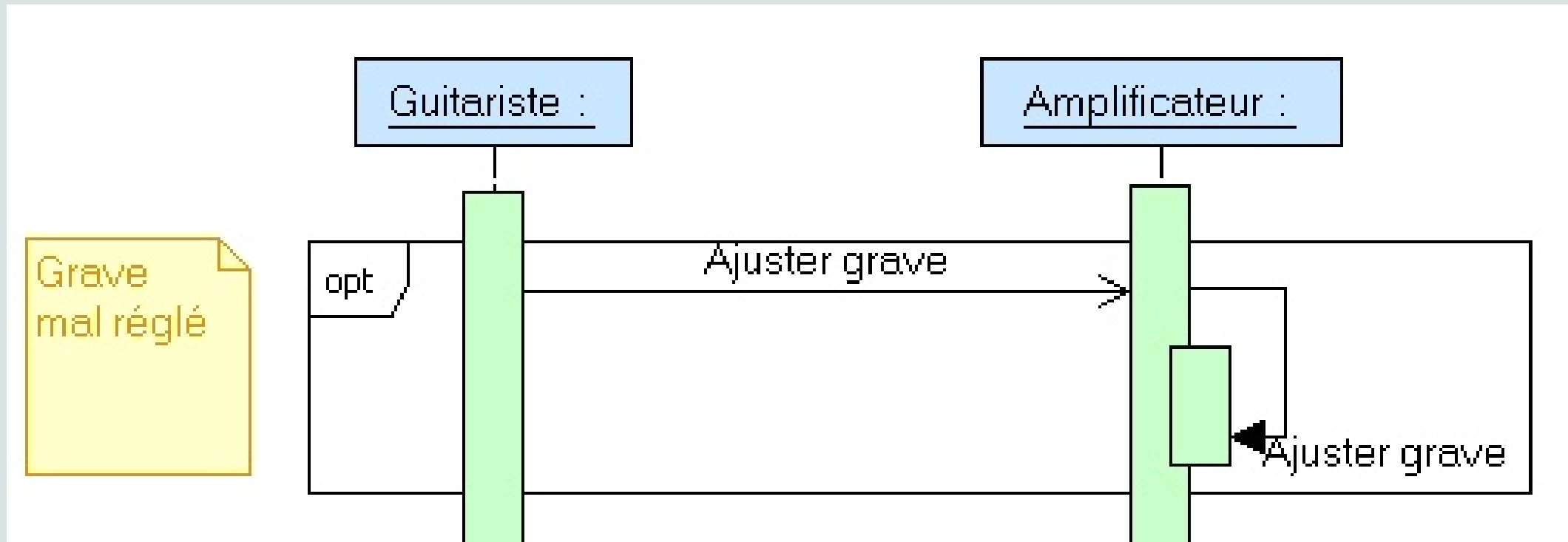
L'opérateur d'option « opt »

L'opérateur option, ou **opt**, comporte un opérande et une condition de garde associée.

Le sous-fragment s'exécute si la condition de garde est vraie et ne s'exécute pas dans le cas contraire.



L'opérateur d'option « opt »

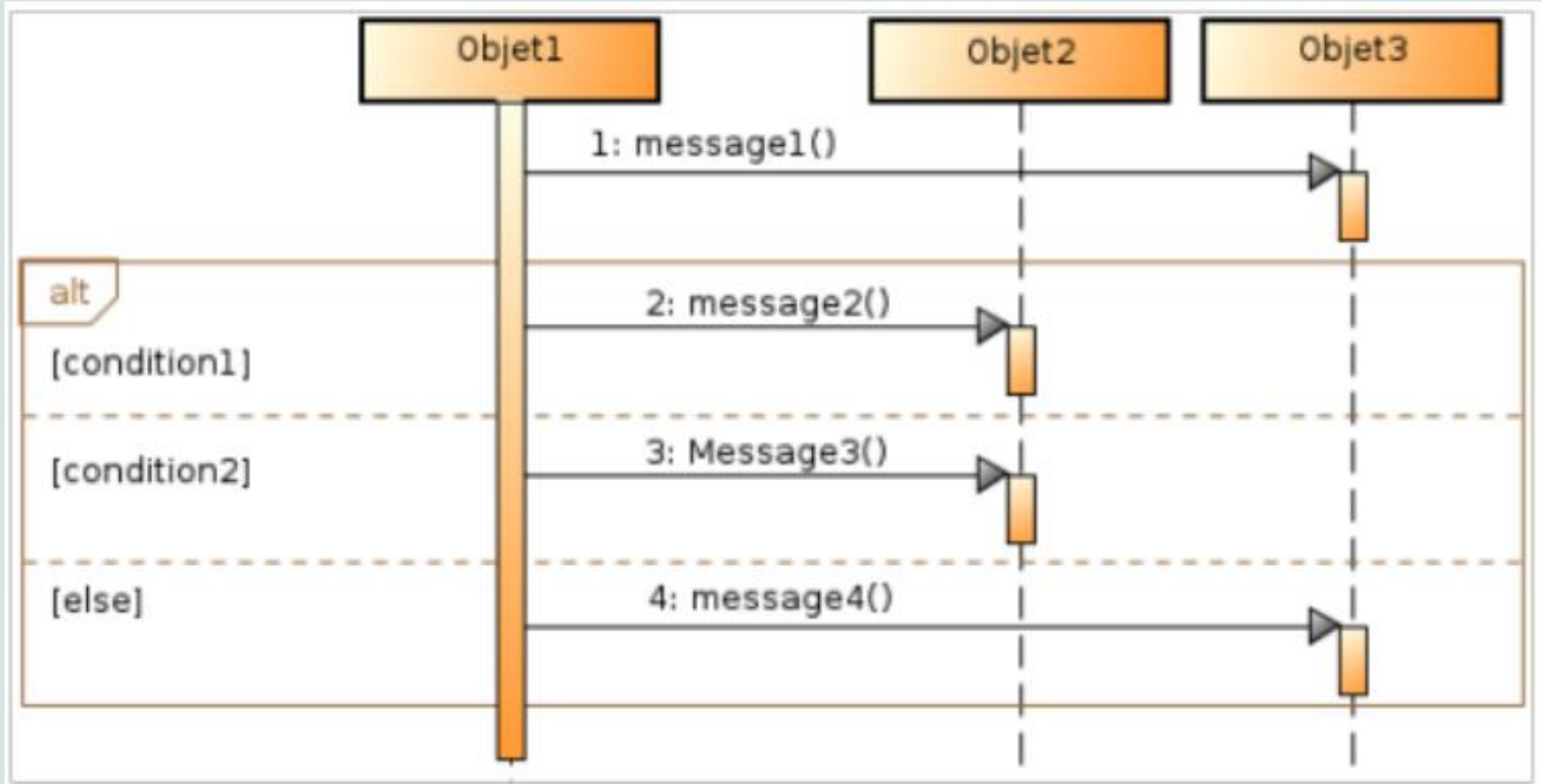


L'opérateur d'alternative « alt »

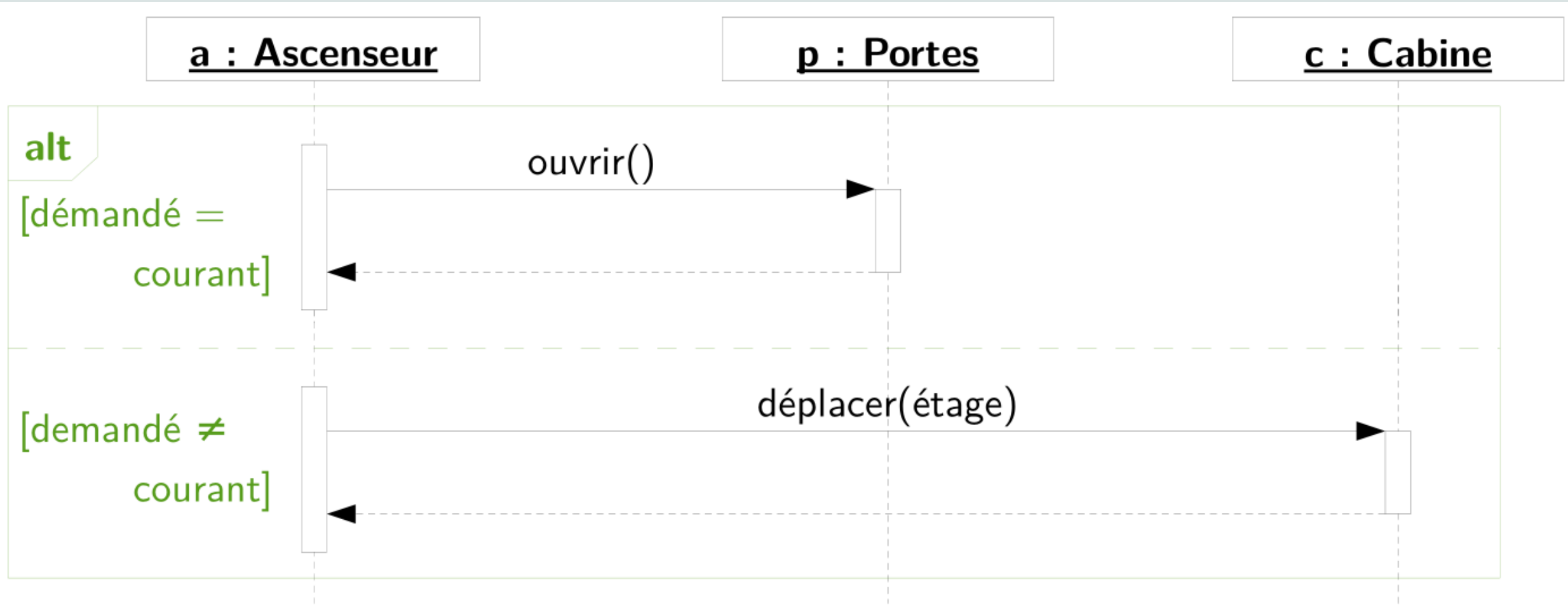
L'opérateur alternative, ou alt, est un opérateur conditionnel possédant plusieurs opérandes séparés par des pointillés. Equivalent à une exécution à choix multiple (condition switch).

Chaque opérande détient une condition de garde. Seul le sous-fragment dont la condition est vraie est exécuté. La condition **else** est exécutée que si aucune autre condition n'est valide.

L'opérateur d'alternative « alt »



L'opérateur d'alternative « alt »



L'opérateur de boucle « loop »

L'opérateur de boucle (loop) exécute la séquence contenue dans le fragment tant que la garde qui lui est associée est vraie.

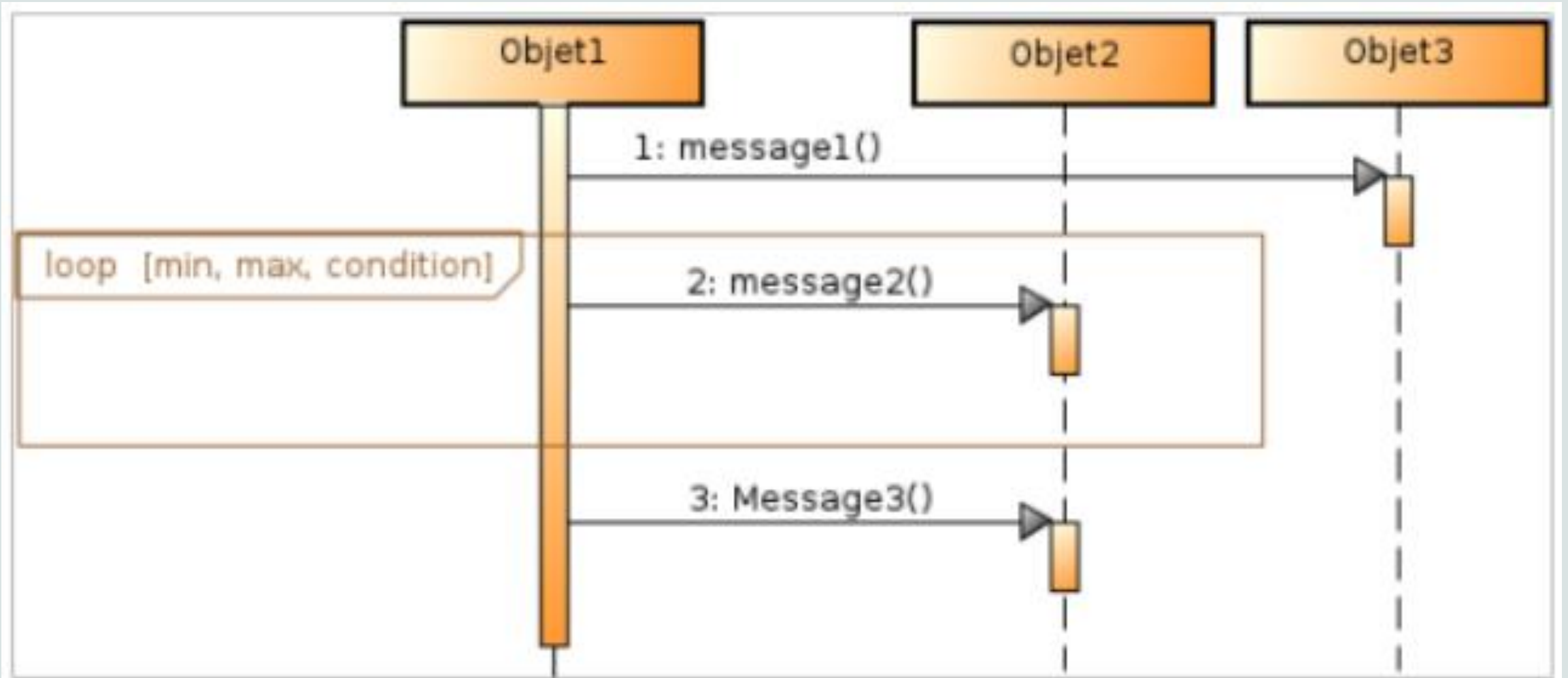
La garde s'écrit de la façon suivante :

loop [min, max, condition]

Chaque paramètre (min, max et condition) est optionnel.

Le contenu du fragment est exécuté min fois, puis continue à s'exécuter tant que la condition et que le nombre d'exécution de la boucle ne dépasse pas max fois.

L'opérateur de boucle « loop »



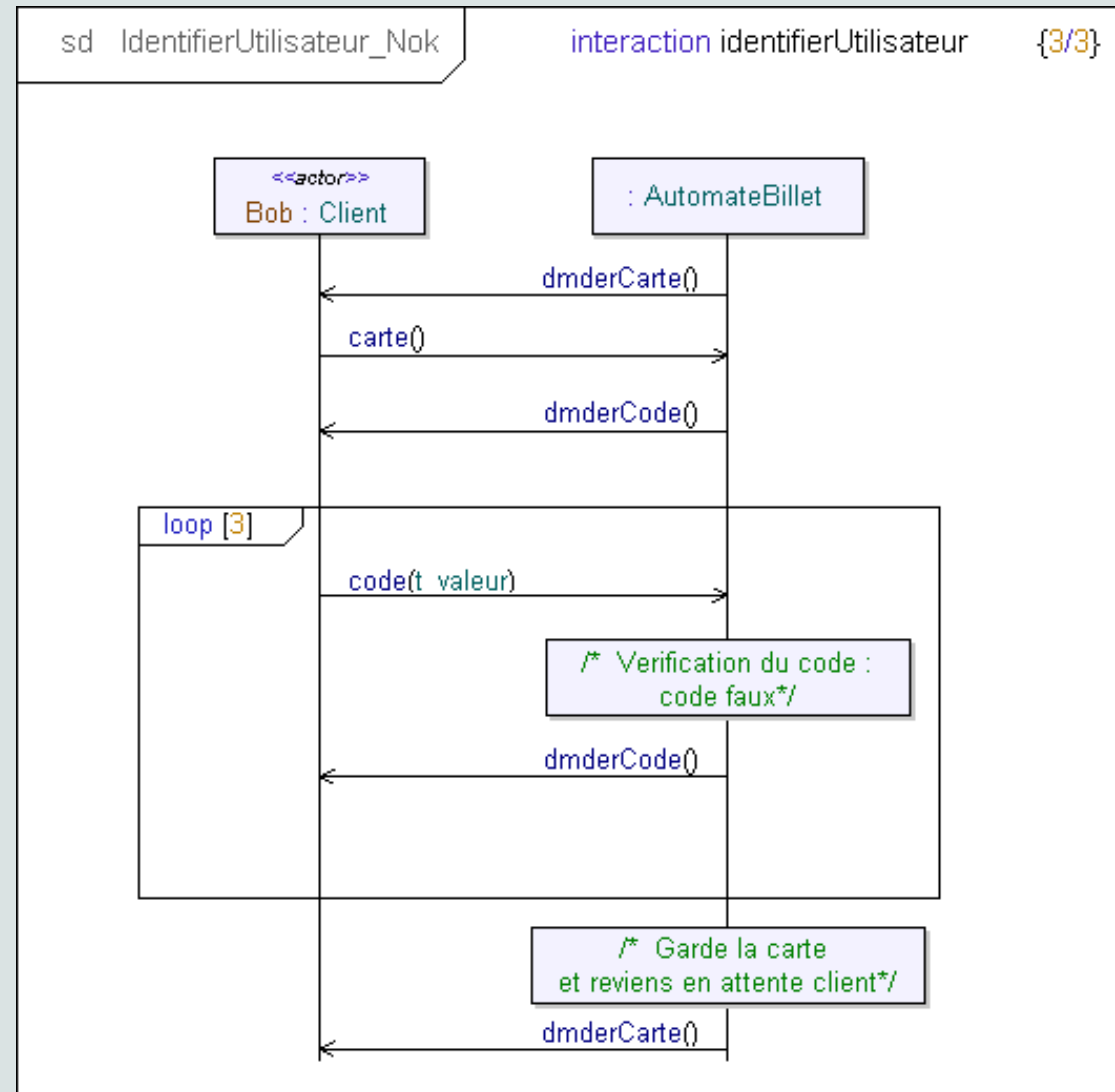
L'opérateur de boucle « loop »

Exemple de gardes :

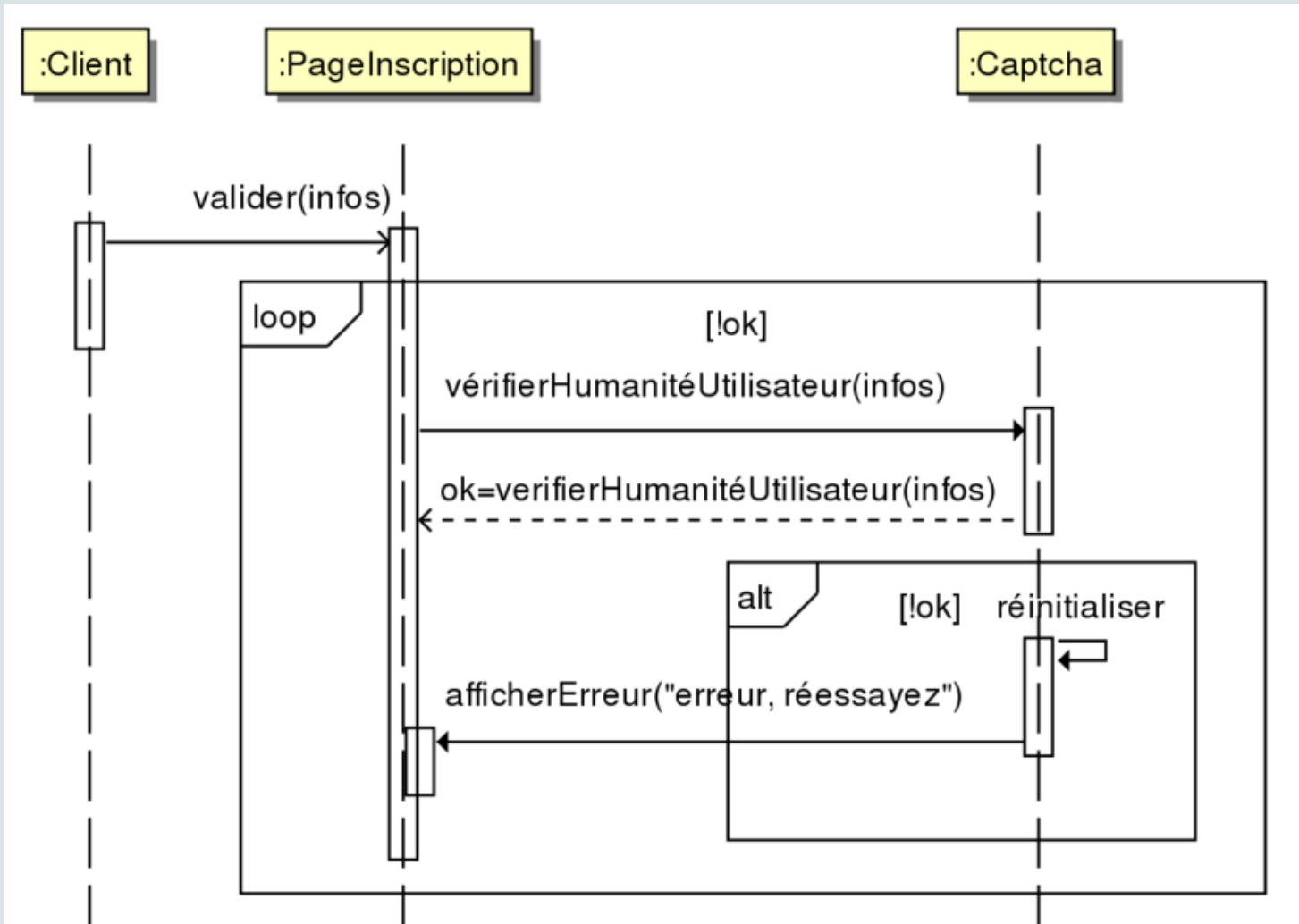
`loop[3]` → La séquence s'exécute 3 fois.

`Loop[1, 3, code=faux]` → La séquence s'exécute 1 fois puis un maximum de 2 autres fois si `code=faux`.

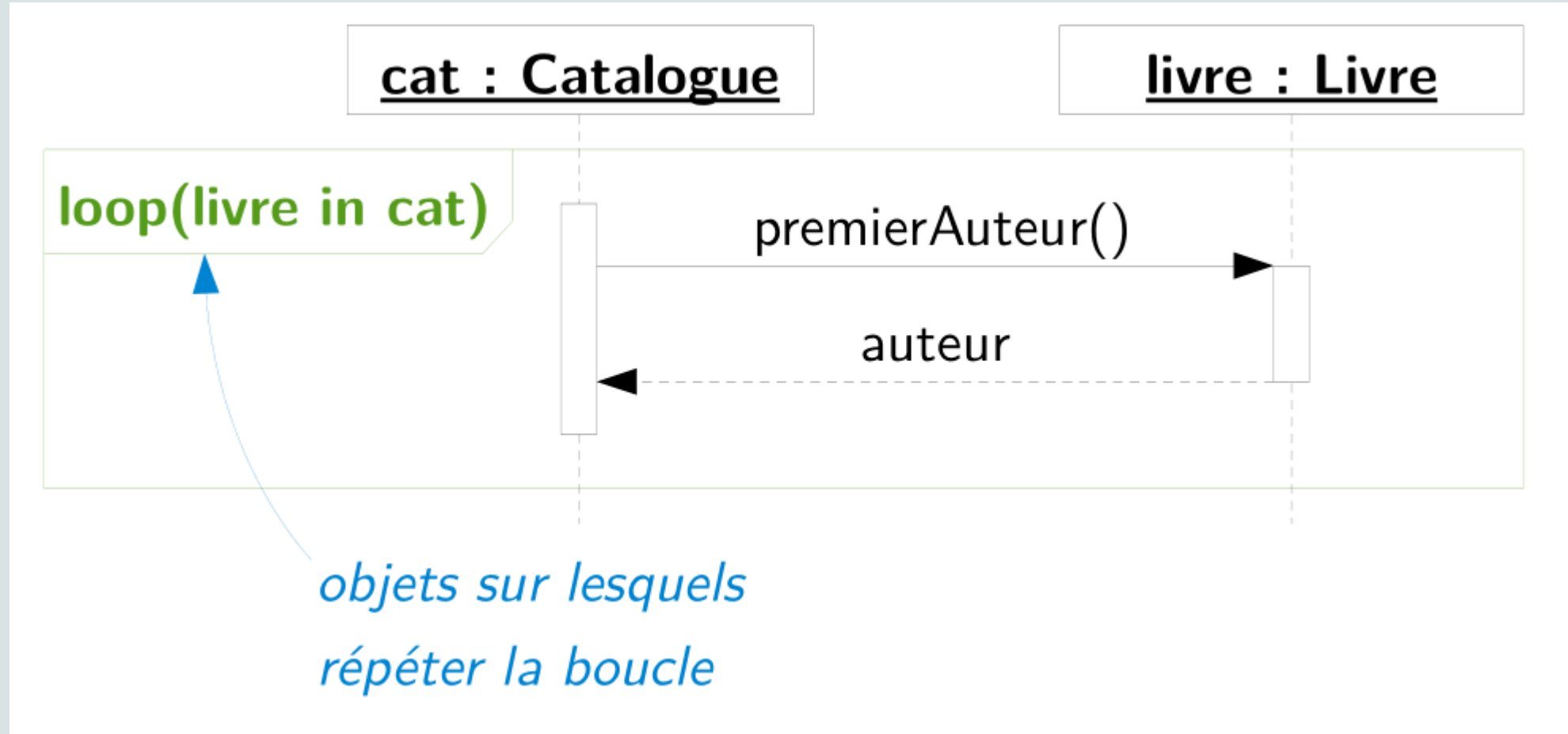
L'opérateur de boucle « loop »



L'opérateur de boucle « loop »



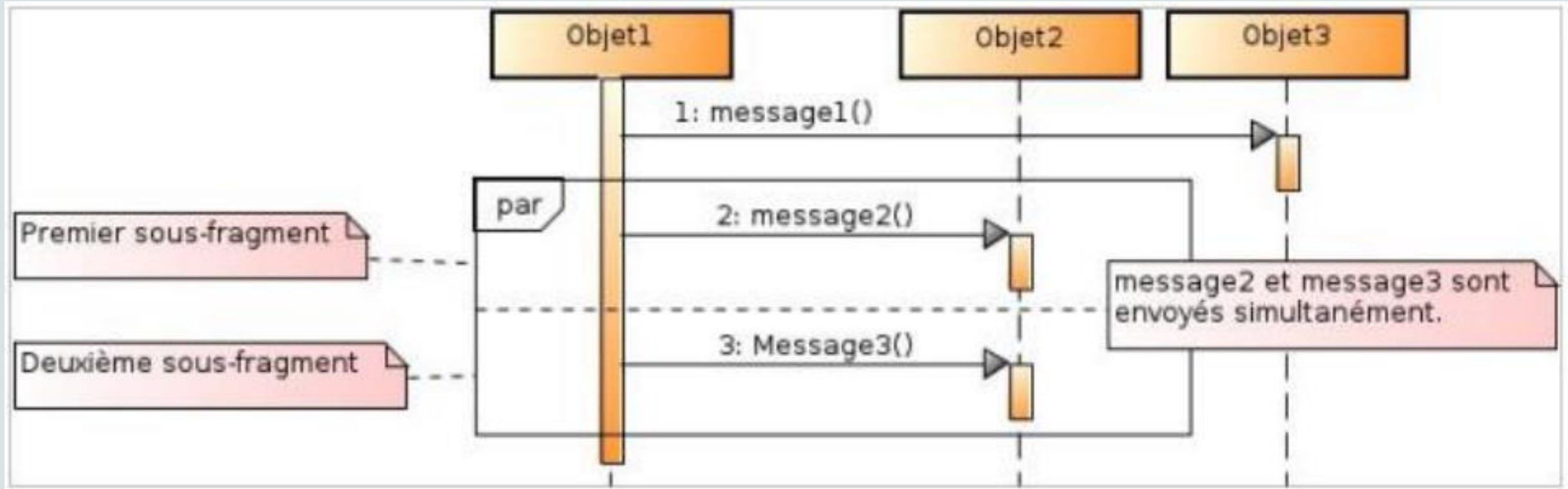
L'opérateur de boucle « loop »



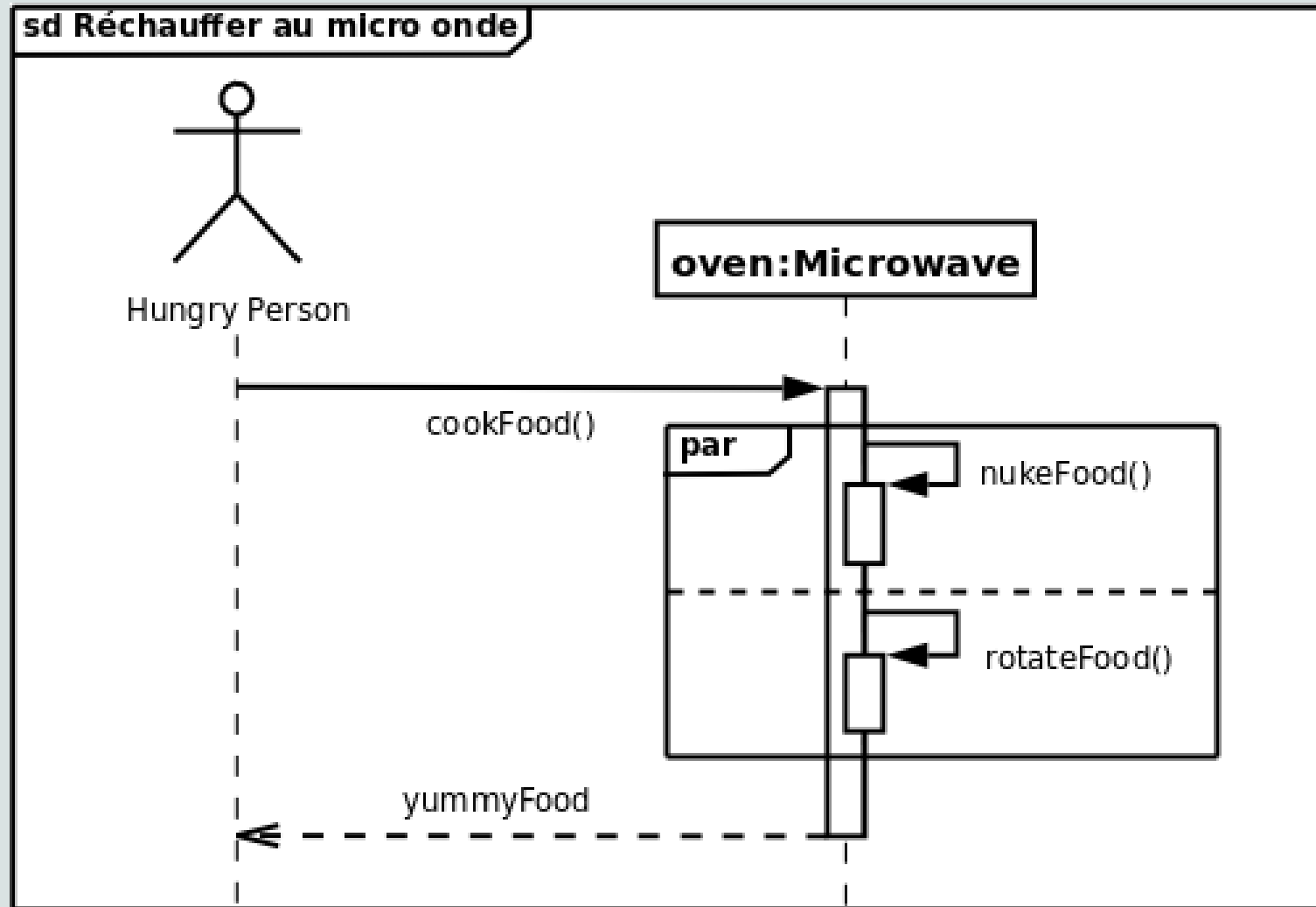
L'opérateur Parallèle « par »

Un fragment d'interaction avec l'opérateur de traitements parallèles (par) contient au moins deux sous fragments (opérandes) séparés par des pointillés qui s'exécutent simultanément (traitements concurrents).

L'opérateur Parallèle « par »



L'opérateur Parallèle « par »



L'opérateur d'interruption « break »

L'opérateur « break » est utilisé dans les fragments combinés qui représentent des scénarios d'exception en quelque sorte. Les interactions de ce fragment seront exécutées à la place des interactions décrites en dessous.

Il y a donc une notion d'interruption du flot « normal » des interactions.

L'opérateur d'interruption « break »

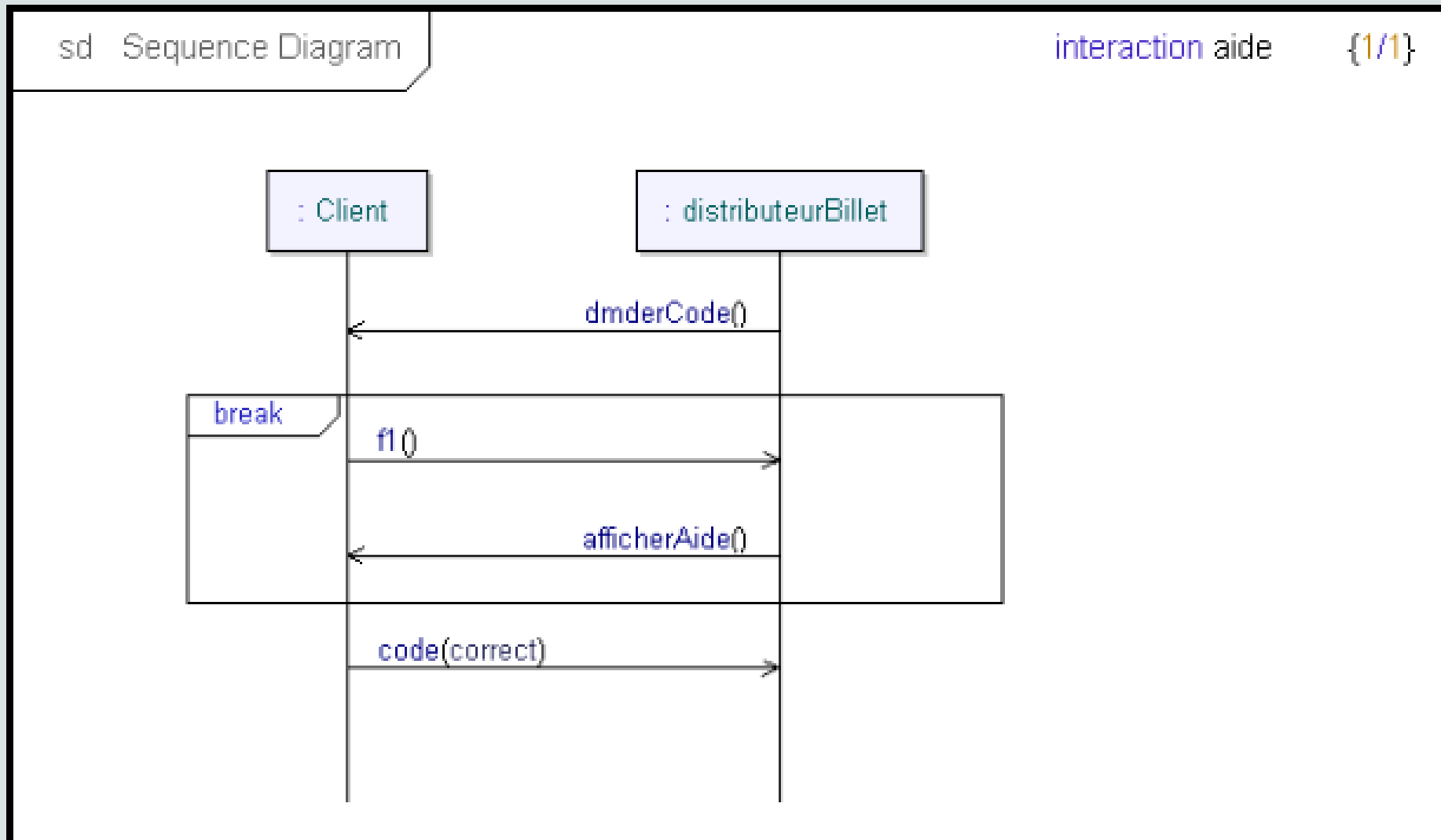
Exemple :

Lorsque le distributeur lui demande son code, l'utilisateur peut choisir de rentrer son code ou de consulter l'aide.

S'il choisit de consulter l'aide (en appuyant sur la touche F1), le flot d'interaction relatif à la saisie du code est interrompu.

Les interactions de l'opérateur break sont « exécutées ».

L'opérateur d'interruption « break »



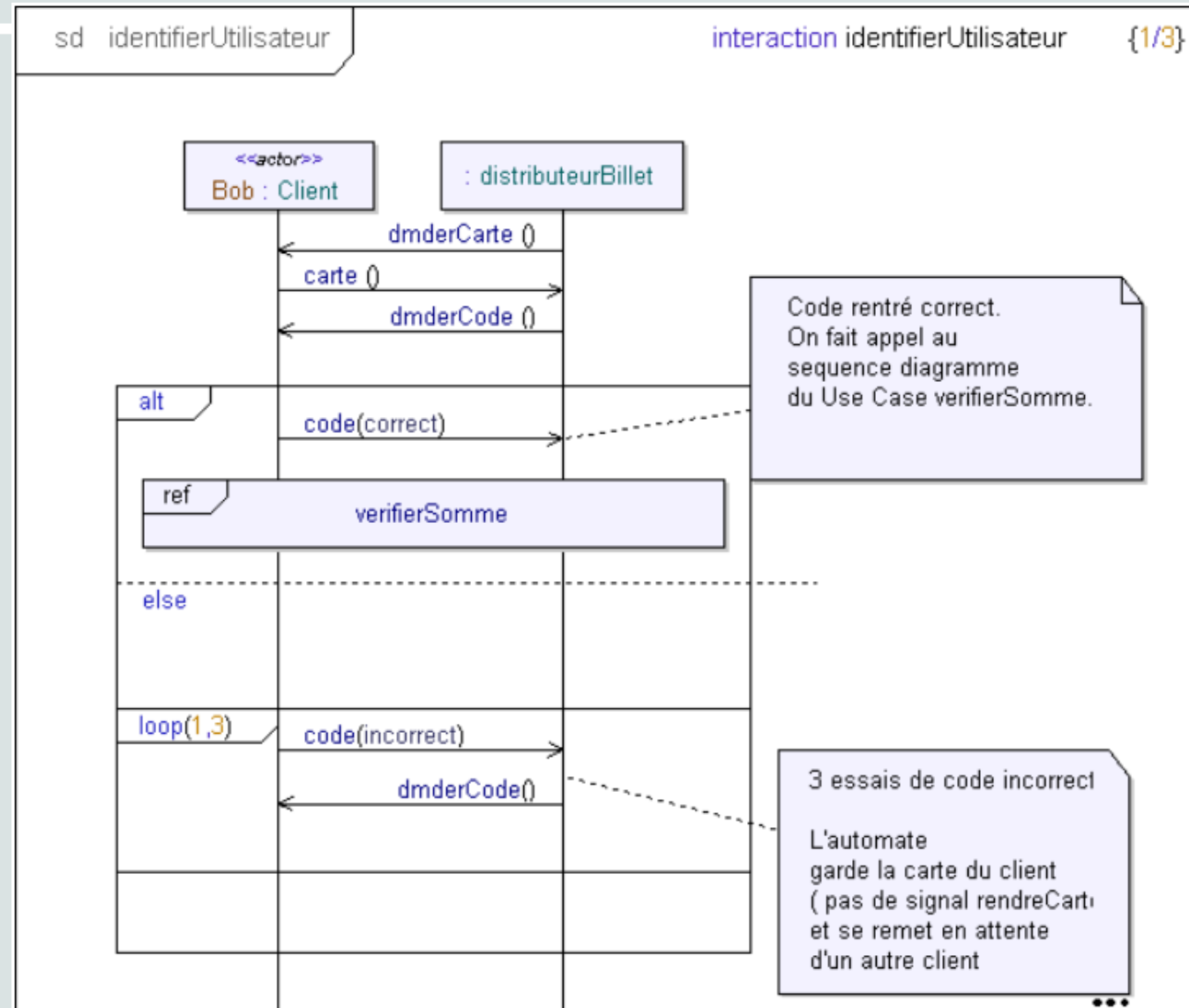
Combiner les opérateurs

Les fragments combinés et leurs opérateurs peuvent être combinés/mixés en vue de décrire des comportements complexes.

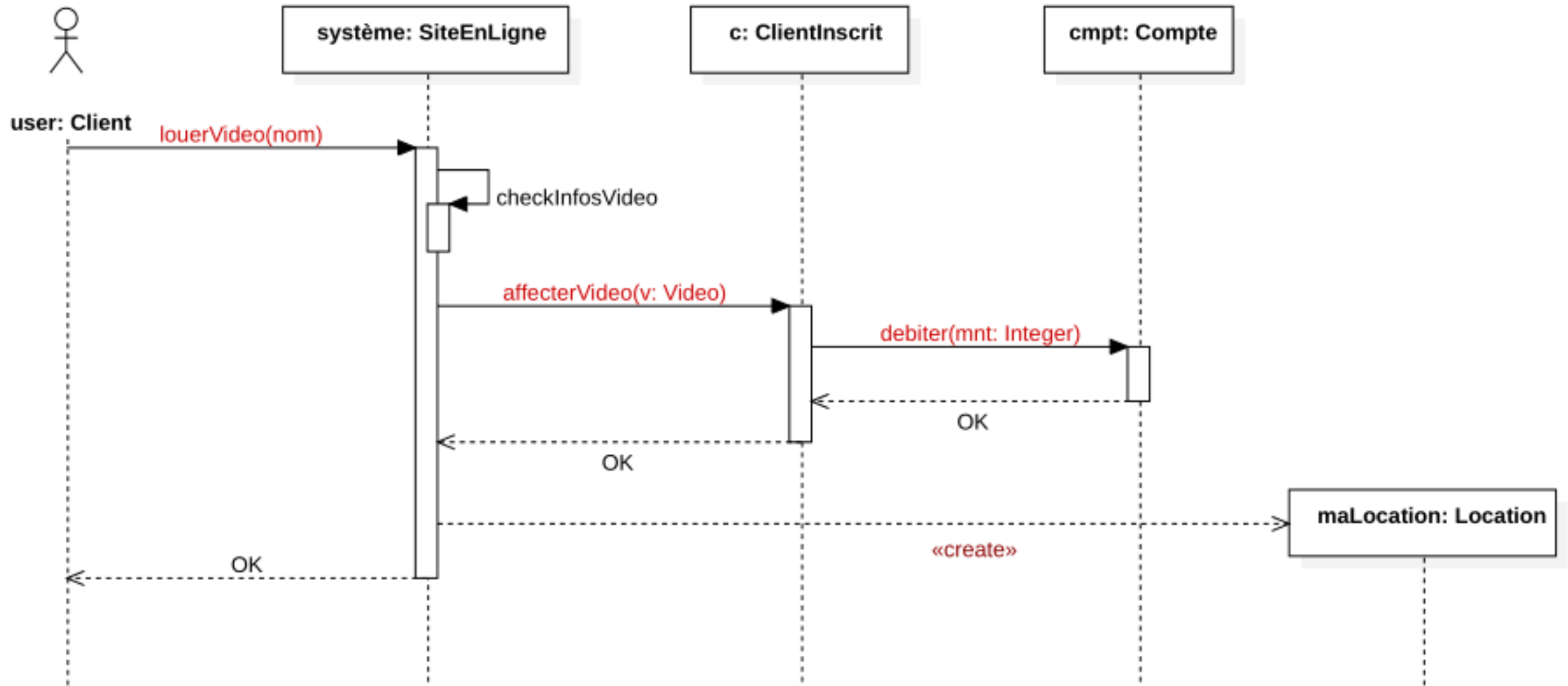
Exemple :

Lorsque l'utilisateur se trompe trois fois de code, la carte est gardée et le distributeur se remet en mode d'attente d'une carte.

Combiner les opérateurs



Exemple 1



Exemple 2

